

Universidad Galileo

Definición del Alcance del Proyecto de Software a desarrollar

GESTIÓN DE PAGOS MENSUALES TRANSPORTES GÉNESIS

Tipo de proyecto:

Mantenimiento

Versión: 1.0

Nombre Autorizador	Firma	Fecha
Jonathan Samuel Villeda Pérez	JONATHAN VILLEDA	12/05/2026
José David Florian	DAVID FLORIÁN	12/05/2026

Contenido

ESPECIFICACIÓN FUNCIONAL - TRANSPORTES GENESIS	5
Sistema de Gestión de Transporte Escolar.....	5
1.1 Introducción y alcance del sistema.....	5
1.1.1 Descripción General	5
1.1.2 Problema que Resuelve.....	5
1.1.3 Objetivos del Proyecto.....	6
1.1.4 Alcance del Sistema	6
1.1.5 Beneficios Esperados	8
1.1.6 Usuarios del Sistema.....	9
1.2 Contexto organizacional y stakeholders.....	9
1.2.1 Modelo Organizacional de Empresa de Transporte Escolar.....	9
1.2.2 Stakeholders del Proyecto	10
1.2.3 Flujo Operativo Actual vs Propuesto	11
1.2.4 Análisis de Impacto en Stakeholders.....	14
1.2.5 Mapa de Empatía por Stakeholder	14
1.3 Análisis de Requerimientos Funcionales.....	15
1.3.1 Requerimientos Funcionales por Módulo.....	15
1.3.2 Matriz de Trazabilidad (Requerimientos ↔ Casos de Uso)	25
1.4 Análisis de Requerimientos No Funcionales.....	26
1.4.1 Requerimientos de Rendimiento (Performance)	26
1.4.2 Requerimientos de Usabilidad.....	27
1.4.3 Requerimientos de Seguridad	28
1.4.4 Requerimientos de Disponibilidad (Availability)	29
1.4.5 Requerimientos de Escalabilidad.....	29
1.4.6 Requerimientos de Mantenibilidad.....	30
1.4.7 Requerimientos de Compatibilidad	30
1.4.8 Requerimientos de Cumplimiento Legal.....	31
1.5 Análisis de Dominio	32
1.5.1 Modelo Conceptual del Negocio.....	32
1.5.2 Reglas de Negocio.....	35
1.5.3 Procesos de Negocio Clave.....	36
1.5.4 Eventos de Dominio	39
1.6 Supuestos, restricciones y dependencias funcionales.....	40

1.6.1 Supuestos del Proyecto	40
1.6.2 Restricciones del Proyecto.....	41
1.6.3 Dependencias Funcionales	43
1.6.4 Riesgos Funcionales	45
1.1.5 Usuarios Finales del Sistema	46
1.1.6 Beneficios Esperados	46
ESPECIFICACIÓN TÉCNICA - TRANSPORTES GENESIS	47
2.1 Arquitectura de referencia.....	47
2.1.1 Diagrama C4 - Nivel 1: Contexto del Sistema	47
2.1.2 Diagrama C4 - Nivel 2: Contenedores.....	48
2.1.3 Diagrama C4 - Nivel 3: Componentes (Web Application).....	50
2.1.4 Estilo Arquitectónico y Justificación	52
2.1.5 Diagrama de Despliegue	54
2.1.6 Principios de Diseño Aplicados.....	56
2.2 Stack tecnológico seleccionado	57
2.2.1 Tecnologías por Capa.....	57
2.2.2 Versiones Exactas de Paquetes NuGet Principales.....	60
2.2.3 Comparación de Alternativas Tecnológicas	61
2.2.4 Trade-offs y Decisiones Técnicas	62
2.3 Requerimientos de infraestructura y entornos	63
2.3.1 Hardware Mínimo y Recomendado (On-Premise).....	63
2.3.2 Requerimientos Cloud (Azure - Recomendado)	64
2.3.3 Diagrama de Red y Topología de Seguridad	65
2.3.4 Entornos de Desarrollo, Pruebas y Producción.....	67
2.3.5 Especificación de Contenedores (Docker) - Opcional	68
2.3.6 CI/CD Pipeline Básico (GitHub Actions).....	70
2.4 Diseño de base de datos	71
2.4.1 Esquema Físico - Tablas Principales.....	71
2.4.2 Diagrama Entidad-Relación (ER)	75
2.4.3 Índices y Estrategias de Optimización	78
2.4.4 Estrategia de Migración de Datos (si aplica)	79
2.5 Diseño de APIs e integración.....	80
2.5.1 Endpoints REST Principales.....	80
2.5.2 SignalR Hub - Geolocalización en Tiempo Real	83

2.5.3 Integración con Servicios Externos	84
2.5.4 Contratos de Interfaz (OpenAPI/Swagger)	86
2.6 Patrones de diseño y buenas prácticas	88
2.6.1 Patrones de Diseño Aplicados	88
2.6.2 Principios SOLID Aplicados	91
2.6.3 Estrategia de Testing.....	91
2.6.4 Buenas Prácticas de Código.....	93
2.7 Plan de implementación y roadmap técnico	94
2.7.1 Fases del Proyecto (Completadas)	94
2.7.2 Roadmap Futuro (v2.0 y posteriores).....	95
2.7.3 Estrategia de DevOps	96
2.8 Análisis de riesgos técnicos y plan de mitigación	97
2.8.1 Matriz de Riesgos Técnicos	97
2.8.2 Plan de Respuesta a Incidentes.....	100
2.8.3 Estrategia de Backup y Recuperación	101
2.8.4 Lessons Learned (Post-Mortem).....	102

ESPECIFICACIÓN FUNCIONAL - TRANSPORTES GENESIS

Sistema de Gestión de Transporte Escolar

Proyecto: Transportes Genesis

Tipo de Documento: Especificación Funcional

Versión: 1.0

Fecha: 2025

Autor: Equipo de Desarrollo TransportesGenesis

Repositorio: <https://github.com/johnsvill/TransportesGenesis>

Rama: dev_intermedia

1.1 Introducción y alcance del sistema

1.1.1 Descripción General

Transportes Genesis es un sistema integral de gestión de transporte escolar desarrollado en .NET 8 con Razor Pages que digitaliza y optimiza las operaciones diarias de empresas de transporte escolar. El sistema proporciona una plataforma centralizada que conecta a todos los actores involucrados en el servicio de transporte: administradores, pilotos, monitores y padres de familia.

1.1.2 Problema que Resuelve

Las empresas de transporte escolar tradicionalmente operan con procesos manuales que generan:

- **Ineficiencia operativa:** Planificación de rutas manual y sin optimización
- **Falta de visibilidad:** Los padres no saben dónde está el bus en tiempo real
- **Desorganización:** Registros en papel de recogidas, asistencias y pagos
- **Baja satisfacción del cliente:** Poca comunicación y transparencia
- **Riesgos de seguridad:** Falta de trazabilidad de quién recoge a cada alumno
- **Costos elevados:** Rutas no optimizadas consumen más combustible
- **Errores humanos:** Pérdida de registros, pagos no confirmados, asistencias mal registradas

1.1.3 Objetivos del Proyecto

Objetivos Generales

1. **Digitalizar** todas las operaciones de transporte escolar en una plataforma web centralizada
2. **Optimizar** las rutas mediante algoritmos de cálculo automático (TSP - Traveling Salesman Problem)
3. **Proporcionar visibilidad en tiempo real** de la ubicación de los buses mediante geolocalización
4. **Mejorar la comunicación** entre la empresa de transporte y los padres de familia
5. **Aumentar la seguridad** con registro detallado de recogidas y asistencias
6. **Reducir costos operativos** mediante rutas optimizadas y procesos automatizados

Objetivos Específicos

1. Implementar un sistema de autenticación y autorización basado en roles
2. Crear un módulo de cálculo automático de rutas optimizadas
3. Desarrollar un sistema de geolocalización en tiempo real con SignalR
4. Proveer un módulo de gestión de pagos para padres de familia
5. Implementar un sistema de confirmación de asistencia diaria
6. Crear un módulo de registro de recogidas para monitores
7. Proporcionar dashboards y reportes para administradores

1.1.4 Alcance del Sistema

Dentro del Alcance (v1.0)

El sistema SÍ incluye:

1. **Gestión de Usuarios y Roles**
 - Login/Logout con ASP.NET Core Identity
 - Roles: Administrador, Piloto, Monitor, Padre de Familia
 - Gestión de perfiles de usuario
 - Cambio de contraseña (obligatorio en primer login)
2. **Gestión de Buses y Asignaciones**
 - CRUD de buses (placa, modelo, capacidad)
 - Asignación de pilotos a buses
 - Asignación de monitores a buses
 - Historial de asignaciones
3. **Gestión de Alumnos**
 - CRUD de alumnos
 - Vinculación con padres de familia
 - Información básica: nombre, fecha de nacimiento, grado

4. **Gestión de Paradas**
 - CRUD de paradas
 - Geolocalización de paradas (latitud, longitud)
 - Direcciones descriptivas
5. **Cálculo y Gestión de Rutas**
 - Cálculo automático de rutas optimizadas (algoritmo TSP)
 - Rutas diferenciadas por turno: Mañana y Tarde
 - Asignación de paradas a rutas con orden secuencial
 - Estimación de horarios por parada
 - Visualización de rutas para pilotos y monitores
6. **Geolocalización en Tiempo Real**
 - Envío de ubicación del bus cada X segundos (SignalR)
 - Mapa en tiempo real para visualizar buses
 - Histórico de ubicaciones (opcional en v1.0)
7. **Módulo de Pagos**
 - Registro de pagos realizados por padres
 - Historial de pagos
 - Estado de pagos (Pendiente, Confirmado)
 - Métodos de pago (efectivo, transferencia, etc.)
8. **Confirmación de Asistencia**
 - Los padres pueden confirmar si el alumno asistirá o no
 - Confirmación diaria
 - Visualización del estado de confirmación
9. **Registro de Recogidas (Monitor)**
 - El monitor registra qué alumnos fueron recogidos en cada parada
 - Prevención de duplicados
 - Registro de fecha/hora y ubicación de recogida
 - Visualización del estado de recogidas
10. **Dashboards y Reportes Básicos**
 - Panel de administración con estadísticas básicas
 - Reportes de pagos
 - Reportes de asistencia

Fuera del Alcance (v1.0)

El sistema **NO incluye** en la versión inicial:

1. **Aplicación móvil nativa** (iOS/Android)
 - Solo interfaz web responsive (funciona en móviles vía navegador)
2. **Notificaciones push**
 - No se envían notificaciones automáticas a dispositivos móviles
 - Posible en versiones futuras con Firebase/OneSignal

3. **Integración con pasarelas de pago en línea**
 - Los pagos se registran manualmente y hay integración con Stripe.
 - Posible en v2.0
4. **Chat en tiempo real** entre usuarios
 - No hay mensajería interna en el sistema
5. **Machine Learning** para predicción de tiempos
 - Predicción de tiempos de llegada con ML no está implementada
 - Posible en versiones futuras
6. **Integración con sistemas contables externos**
 - No hay conexión con QuickBooks, SAP, etc.
7. **Gestión de mantenimiento de buses**
 - Registro de mantenimientos preventivos/correctivos no incluido en v1.0
8. **Módulo de quejas y reclamos formal**
 - No hay sistema de tickets de soporte en v1.0
9. **Análisis avanzado de datos (BI)**
 - Dashboards básicos únicamente; no hay herramientas de Business Intelligence
10. **Multi-idioma**
 - El sistema está únicamente en español

1.1.5 Beneficios Esperados

Para la Empresa de Transporte

-  Reducción del 20-30% en costos de combustible gracias a rutas optimizadas
-  Ahorro de 10-15 horas/semana en tareas administrativas
-  Mayor control operativo con visibilidad en tiempo real de toda la flota
-  Reducción de errores en registros de pagos y asistencias
-  Mejora en la imagen corporativa al ofrecer tecnología moderna

Para los Pilotos

-  Rutas claras y optimizadas sin necesidad de planificación manual
-  Información actualizada de alumnos que asistirán cada día
-  Interfaz simple para ver su ruta diaria

Para los Monitores

-  Proceso digitalizado de registro de recogidas (adiós al papel)
-  Prevención de errores con validaciones automáticas
-  Interfaz intuitiva para marcar asistencias en tiempo real

Para los Padres de Familia

- ☒ Tranquilidad al ver dónde está el bus en tiempo real
- ☒ Comunicación efectiva al confirmar asistencia de su hijo
- ☒ Transparencia en el estado de pagos
- ☒ Accesibilidad desde cualquier dispositivo con navegador

1.1.6 Usuarios del Sistema

Rol	Descripción	Cantidad Estimada	Funcionalidades Principales
Administrador	Dueño o gerente de la empresa de transporte	1-3 por empresa	Gestión completa del sistema, reportes, configuración
Piloto	Conductor del bus	5-50 por empresa	Ver su ruta diaria, actualizar ubicación
Monitor	Acompañante en el bus que supervisa a los alumnos	5-50 por empresa	Ver ruta, registrar recogidas, marcar asistencias
Padre de Familia	Tutor del alumno	50-500 por empresa	Ver ubicación del bus, confirmar asistencia, ver pagos

☒ 1.1 INTRODUCCIÓN Y ALCANCE DEL SISTEMA - COMPLETADO

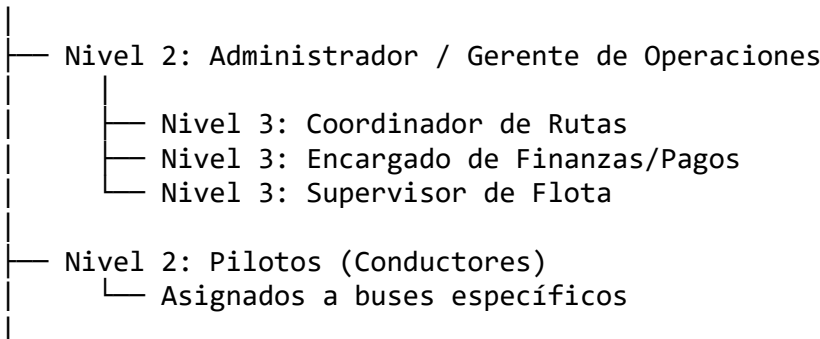
1.2 Contexto organizacional y stakeholders

1.2.1 Modelo Organizacional de Empresa de Transporte Escolar

[INSTRUCCIONES PARA DIAGRAMA ORGANIZACIONAL]

Título: Estructura Organizacional Típica - Empresa de Transporte Escolar

Nivel 1: Dirección General / Dueño



- └ Nivel 2: Monitores
 - └ Asignados a buses específicos

Relación externa:

- Padres de Familia → Clientes del servicio
- Alumnos → Usuarios finales del transporte

Interacción con el sistema:

- Administrador gestiona todo
- Pilotos y Monitores operan el servicio diario
- Padres consultan información y realizan pagos

1.2.2 Stakeholders del Proyecto

Stakeholders Primarios (Usuarios Directos)

Stakeholder	Rol en el Negocio	Interés en el Sistema	Influencia	Expectativas
Dueño de la empresa	Inversionista principal	Alto	● Crítica	ROI, eficiencia, reducción de costos
Administrador	Gerente de operaciones	Alto	● Crítica	Sistema completo, confiable, reportes precisos
Pilotos	Operadores de buses	Medio	● Media	Interfaz simple, rutas claras
Monitores	Supervisores de alumnos	Medio	● Media	Registro rápido, fácil de usar en movimiento
Padres de Familia	Clientes pagadores	Alto	● Baja	Visibilidad, transparencia, seguridad de sus hijos
Alumnos	Usuarios finales (pasivos)	Bajo	● Baja	No interactúan directamente con el sistema

Stakeholders Secundarios

Stakeholder	Interés	Influencia	Expectativas
Instituciones Educativas	Coordinación con servicio de transporte	● Media	Puntualidad, comunicación con padres
Autoridades de Tránsito	Regulación y permisos	● Baja	Cumplimiento de normativas (no aplica al software directamente)
Proveedores de Tecnología	Azure, Google Maps API	● Media	Pago puntual de servicios, uso correcto de APIs
Equipo de Desarrollo	Implementadores	● Crítica	Requerimientos claros, feedback oportuno

1.2.3 Flujo Operativo Actual vs Propuesto

Proceso Actual (Manual)

Flujo: Planificación de Rutas

[Administrador]

- ↓
- 1. Recibe lista de alumnos y direcciones en Excel
- ↓
- 2. Imprime mapa y marca ubicaciones manualmente
- ↓
- 3. Traza rutas "a ojo" intentando optimizar
- ↓
- 4. Comunica rutas a pilotos por WhatsApp o papel
- ↓
- 5. Pilotos memorizan o anotan en papel

Problemas:

- Rutas no optimizadas (más tiempo y combustible)
- Cambios no se comunican eficientemente
- Sin visibilidad de quién recoge a quién

Flujo: Registro de Recogidas

[Monitor]

- ↓
- 1. Lleva lista impresa de alumnos por parada
- ↓
- 2. Marca con lápiz quién subió al bus
- ↓
- 3. Al final del día, entrega lista al administrador
- ↓
- 4. Administrador transcribe a Excel (doble trabajo)

Problemas:

- Papeles se pierden o mojan
 - Caligrafía ilegible
 - Retraso en información (no es en tiempo real)
-

Flujo: Control de Pagos

[Padre de Familia]

- ↓
- 1. Hace depósito/transferencia bancaria
- ↓
- 2. Toma foto del comprobante, envía por WhatsApp al administrador
- ↓
- 3. Administrador anota en Excel manualmente
- ↓
- 4. Padre pregunta si se confirmó el pago (días después)

Problemas:

- Desorganización en WhatsApp (mensajes se pierden)
 - No hay trazabilidad clara
 - Padre no tiene acceso a su historial de pagos
-

Proceso Propuesto (Digitalizado con Transportes Genesis)

Flujo: Planificación de Rutas

[Administrador en el sistema]

- ↓
- 1. Ingresa alumnos y paradas con geolocalización
- ↓
- 2. Sistema calcula ruta óptima automáticamente (algoritmo TSP)
- ↓
- 3. Asigna ruta al bus y al piloto
- ↓
- 4. Piloto/Monitor acceden a su ruta desde su panel
- ↓
- 5. Ven mapa con orden de paradas y horarios estimados

Beneficios:

- Rutas optimizadas (ahorro 20-30% combustible)
 - Cambios instantáneos (todos actualizados)
 - Visibilidad total
-

Flujo: Registro de Recogidas

[Monitor en el sistema (tablet/móvil)]

- ↓
1. Ve su ruta con lista de alumnos por parada
 - ↓
 2. En cada parada, marca checkboxes de quién subió
 - ↓
 3. Sistema registra automáticamente:
 - Fecha/hora
 - Ubicación GPS
 - Usuario que registró
 - ↓
 4. Administrador ve reportes en tiempo real

Beneficios:

- Digitalizado, sin papel
 - Información en tiempo real
 - Prevención de duplicados
-

Flujo: Control de Pagos

[Padre de Familia en el sistema]

- ↓
1. Hace depósito/transferencia bancaria
 - ↓
 2. Entra al sistema, sección "Mis Pagos"
 - ↓
 3. Ingresa:
 - Monto
 - Método (transferencia/efectivo)
 - Comprobante (sube foto)
 - ↓
 4. Administrador recibe notificación, valida y confirma
 - ↓
 5. Padre ve estado actualizado: "Confirmado"
 - ↓
 6. Historial de pagos siempre accesible

Beneficios:

- Padre tiene acceso a su historial
- Trazabilidad completa
- Organización centralizada (no WhatsApp)

1.2.4 Análisis de Impacto en Stakeholders

Stakeholder	Impacto del Sistema	Cambios en su Trabajo	Resistencia Esperada	Estrategia de Mitigación
Administrador	● Positivo Alto	Digitalización total, más tiempo para estrategia	● Media (curva de aprendizaje)	Capacitación dedicada, soporte 1:1
Pilotos	● Positivo Medio	Menos confusión, rutas claras	● Media (algunos no son tech-savvy)	Interfaz muy simple, entrenamiento práctico
Monitores	● Positivo Alto	Adiós al papel, proceso más rápido	● Media (acostumbrados al papel)	Onboarding con tablet, práctica guiada
Padres	● Positivo Alto	Mayor tranquilidad y control	● Baja (la mayoría usa smartphones)	Tutorial en video, soporte por WhatsApp

1.2.5 Mapa de Empatía por Stakeholder

Administrador

¿Qué piensa y siente? - “Necesito controlar todo desde un solo lugar” - “No puedo perder tiempo transcribiendo datos manualmente” - “Quiero que los padres estén satisfechos para retenerlos como clientes”

¿Qué ve? - Excel desorganizado - WhatsApp lleno de mensajes de padres y pilotos - Papeles con listas de recogidas acumuladas

¿Qué dice y hace? - “¿Ya registraron las recogidas de hoy?” - “¿Confirmaron los pagos de esta semana?” - Pasa horas consolidando información

Pains (Dolores): - Pérdida de información - Errores humanos - Falta de reportes consolidados

Gains (Ganancias esperadas): - Ahorro de 10+ horas/semana - Reportes automáticos - Control total en un solo sistema

Padre de Familia

¿Qué piensa y siente? - “¿Dónde está el bus? ¿Ya pasó?” - “¿Recogieron a mi hijo?” - “¿Por qué no me confirman si mi pago fue recibido?”

¿Qué ve? - Su hijo esperando en la calle - Falta de comunicación de la empresa

¿Qué dice y hace? - Llama a la empresa preguntando por el bus - Envía comprobantes de pago por WhatsApp - Pregunta varias veces si el pago fue confirmado

Pains (Dolores): -Ansiedad por no saber dónde está el bus - Falta de transparencia en pagos

Gains (Ganancias esperadas): -Ver el bus en tiempo real (tranquilidad) - Historial de pagos siempre accesible - Confirmar asistencia de su hijo fácilmente

1.3 Análisis de Requerimientos Funcionales

1.3.1 Requerimientos Funcionales por Módulo

RF-01: Módulo de Autenticación y Autorización

ID	Requerimiento	Prioridad	Actor	Criterio de Aceptación
RF-01.1	El sistema debe permitir login con email y contraseña	● Crítica	Todos	Usuario accede con credenciales válidas
RF-01.2	El sistema debe soportar 4 roles: Administrador, Piloto, Monitor, Padre de Familia	● Crítica	Sistema	Cada usuario solo ve funcionalidades de su rol
RF-01.3	El sistema debe forzar cambio de contraseña en primer login	● Alta	Todos	Usuario no puede acceder hasta cambiar contraseña
RF-01.4	El sistema debe permitir logout	● Crítica	Todos	Sesión se cierra correctamente
RF-01.5	El sistema debe redirigir a cada rol a su página principal después del login	● Alta	Sistema	Administrador → Admin panel, Piloto → MiRuta, Monitor → MiRuta, Padre → Mapa
RF-01.6	El sistema debe mostrar mensaje de error si credenciales son inválidas	● Alta	Todos	Mensaje: "Email o contraseña incorrectos"

RF-02: Módulo de Gestión de Buses

ID	Requerimiento	Prioridad	Actor	Criterio de Aceptación
RF-02.1	El administrador debe poder crear un bus con: placa, modelo, capacidad	● Crítica	Administrador	Bus se guarda en BD con validaciones
RF-02.2	El administrador debe poder listar todos los buses	● Crítica	Administrador	Tabla con todos los buses y acciones (editar, eliminar)
RF-02.3	El administrador debe poder editar un bus	● Alta	Administrador	Cambios se reflejan en BD
RF-02.4	El administrador debe poder eliminar un bus (soft delete)	● Alta	Administrador	Bus no se muestra en listados pero se mantiene en BD
RF-02.5	El sistema debe validar que la placa sea única	● Alta	Sistema	Error si placa ya existe
RF-02.6	El administrador debe poder asignar un piloto a un bus	● Crítica	Administrador	Registro en tabla AsignacionPilotoBus
RF-02.7	El administrador debe poder ver historial de asignaciones de un bus	● Media	Administrador	Listado de pilotos que han operado el bus

RF-03: Módulo de Gestión de Alumnos

ID	Requerimiento	Prioridad	Actor	Criterio de Aceptación
RF-03.1	El administrador debe poder crear un alumno con: nombre, apellido, fecha de nacimiento, grado	● Crítica	Administrador	Alumno se guarda en BD
RF-03.2	El administrador debe poder asignar un alumno a un padre de familia	● Crítica	Administrador	Relación en tabla AlumnoPadreFamilia
RF-03.3	El administrador debe poder asignar un alumno a una parada	● Crítica	Administrador	Registro en AsignacionAlumnoParada
RF-03.4	El administrador debe poder listar todos los alumnos	● Crítica	Administrador	Tabla con alumnos y acciones
RF-03.5	El administrador debe poder editar un alumno	● Alta	Administrador	Cambios se reflejan en BD
RF-03.6	El administrador debe poder eliminar un alumno (soft delete)	● Alta	Administrador	Alumno no se muestra, pero se mantiene en BD
RF-03.7	El sistema debe validar que la fecha de nacimiento sea coherente (alumno entre 3-18 años)	● Media	Sistema	Error si fecha no es válida

RF-04: Módulo de Gestión de Paradas

ID	Requerimiento	Prioridad	Actor	Criterio de Aceptación
RF-04.1	El administrador debe poder crear una parada con: nombre, dirección, latitud, longitud	● Crítica	Administrador	Parada se guarda en BD
RF-04.2	El sistema debe permitir seleccionar la ubicación en un mapa interactivo	● Alta	Administrador	Click en mapa rellena latitud/longitud automáticamente
RF-04.3	El administrador debe poder listar todas las paradas	● Crítica	Administrador	Tabla con paradas y acciones
RF-04.4	El administrador debe poder editar una parada	● Alta	Administrador	Cambios se reflejan en BD
RF-04.5	El administrador debe poder eliminar una parada (soft delete)	● Alta	Administrador	Parada no se muestra pero se mantiene en BD
RF-04.6	El sistema debe validar que latitud esté entre -90 y 90	● Alta	Sistema	Error si latitud inválida
RF-04.7	El sistema debe validar que longitud esté entre -180 y 180	● Alta	Sistema	Error si longitud inválida

RF-05: Módulo de Cálculo de Rutas

ID	Requerimiento	Prioridad	Actor	Criterio de Aceptación
RF-05.1	El administrador debe poder calcular una ruta automáticamente e para un bus y turno específico	● Crítica	Administrador	Sistema ejecuta algoritmo TSP y genera ruta óptima
RF-05.2	El sistema debe calcular rutas diferenciadas para turno Mañana y Tarde	● Crítica	Sistema	Dos rutas independientes por bus
RF-05.3	El sistema debe asignar un orden secuencial a cada parada en la ruta	● Crítica	Sistema	Campo Orden en ParadaRuta
RF-05.4	El sistema debe calcular horario estimado para cada parada	● Alta	Sistema	Campo HorarioEstimado basado en distancia y velocidad promedio
RF-05.5	El sistema debe permitir al administrador editar manualmente el orden de paradas	● Media	Administrador	Drag-and-drop o campo numérico editable
RF-05.6	El sistema debe recalcular la ruta si se agregan o eliminan paradas	● Media	Sistema	Ruta se actualiza automáticamente
RF-05.7	El sistema debe mostrar la ruta calculada en un mapa	● Alta	Administrador	Mapa con línea que conecta paradas en orden

RF-06: Módulo de Geolocalización en Tiempo Real

ID	Requerimiento	Prioridad	Actor	Criterio de Aceptación
RF-06.1	El piloto debe poder enviar su ubicación GPS en tiempo real	● Crítica	Piloto	Ubicación se envía vía SignalR cada 10 segundos
RF-06.2	El padre de familia debe poder ver el bus en un mapa en tiempo real	● Crítica	Padre	Mapa muestra ícono del bus en posición actual
RF-06.3	El sistema debe mostrar la ruta completa con paradas en el mapa	● Alta	Padre, Piloto	Mapa con marcadores en cada parada
RF-06.4	El sistema debe actualizar la posición del bus sin recargar la página	● Crítica	Sistema	SignalR actualiza mapa dinámicamente
RF-06.5	El sistema debe mostrar mensaje si el bus no está transmitiendo ubicación	● Alta	Padre	Mensaje: "El bus no está en operación actualmente"
RF-06.6	El sistema debe registrar historial de ubicaciones (opcional)	● Baja	Sistema	Tabla HistorialUbicaciones (futuro)

RF-07: Módulo de Pagos

ID	Requerimiento	Prioridad	Actor	Criterio de Aceptación
RF-07.1	El padre debe poder registrar un pago con: monto, fecha, método, comprobante	● Crítica	Padre	Pago se guarda con estado "Pendiente"
RF-07.2	El padre debe poder subir una imagen del comprobante de pago	● Alta	Padre	Imagen se guarda en servidor o Azure Blob
RF-07.3	El padre debe poder ver su historial de pagos	● Crítica	Padre	Tabla con todos sus pagos y estados
RF-07.4	El administrador debe poder listar todos los pagos pendientes	● Crítica	Administrador	Tabla filtrada por estado "Pendiente"
RF-07.5	El administrador debe poder confirmar un pago	● Crítica	Administrador	Estado cambia a "Confirmado"
RF-07.6	El administrador debe poder rechazar un pago con motivo	● Alta	Administrador	Estado cambia a "Rechazado", se registra motivo

RF-07.7	El sistema debe permitir filtrar pagos por alumno, fecha, estado	● Media	Administrador	Filtros funcionales en listado
RF-07.8	El sistema debe calcular el total de ingresos en un rango de fechas	● Media	Administrador	Reporte con suma de pagos confirmados

RF-08: Módulo de Confirmación de Asistencia

ID	Requerimiento	Prioridad	Actor	Criterio de Aceptación
RF-08.1	El padre debe poder confirmar diariamente si su hijo asistirá	● Crítica	Padre	Registro en tabla ConfirmacionAsistencia
RF-08.2	El sistema debe mostrar el estado de confirmación para el día actual	● Alta	Padre	Indicador visual: "Confirmado" / "No confirmado"
RF-08.3	El piloto/monitor debe poder ver qué alumnos confirmaron asistencia	● Crítica	Piloto, Monitor	Lista con checkmarks de confirmaciones
RF-08.4	El sistema debe permitir cambiar la confirmación hasta 1 hora antes del	● Media	Padre	Después de la hora límite, no permite cambios

	inicio de ruta			
RF-08.5	El sistema debe enviar recordatorio si el padre no ha confirmado (futuro)	 Baja	Sistema	Notificación o email automático

RF-09: Módulo de Registro de Recogidas (Monitor)

ID	Requerimiento	Prioridad	Actor	Criterio de Aceptación
RF-09.1	El monitor debe poder ver su ruta diaria con lista de alumnos por parada	 Crítica	Monitor	Página Registrar Recogidas con ruta y alumnos
RF-09.2	El monitor debe poder marcar qué alumnos fueron recogidos en cada parada	 Crítica	Monitor	Checkboxes por alumno, botón "Guardar"
RF-09.3	El sistema debe registrar fecha/hora y ubicación GPS al guardar recogidas	 Alta	Sistema	Campos automáticos en Registro Recogida
RF-09.4	El sistema debe prevenir registros duplicados del mismo alumno en el mismo día	 Crítica	Sistema	Validación: error si ya existe registro
RF-09.5	El monitor debe poder ver qué alumnos ya fueron registrados	 Alta	Monitor	Checkboxes pre-marcados si ya hay registro
RF-09.6	El administrador debe poder ver	 Alta	Administrador	Reporte con filtros

	reportes de recogidas por fecha y bus			
--	---------------------------------------	--	--	--

RF-10: Dashboard y Reportes

ID	Requerimiento	Prioridad	Actor	Criterio de Aceptación
RF-10.1	El administrador debe ver un dashboard con estadísticas generales	● Alta	Administrador	Tarjetas con: total de buses, alumnos, pagos pendientes, etc.
RF-10.2	El administrador debe poder generar reporte de pagos por rango de fechas	● Alta	Administrador	PDF o Excel con detalle de pagos
RF-10.3	El administrador debe poder generar reporte de asistencia por alumno	● Media	Administrador	Reporte con % de asistencia
RF-10.4	El administrador debe poder generar reporte de recogidas por bus	● Media	Administrador	Reporte con detalle de recogidas diarias
RF-10.5	El sistema debe permitir exportar reportes a Excel	● Media	Administrador	Botón "Exportar a Excel" funcional

1.3.2 Matriz de Trazabilidad (Requerimientos ↔ Casos de Uso)

Caso de Uso	Requerimientos Relacionados	Prioridad	Complejidad
CU-01: Login	RF-01.1, RF-01.2, RF-01.5, RF-01.6	● Crítica	● Baja
CU-02: Cambiar contraseña	RF-01.3	● Alta	● Baja
CU-03: Crear bus	RF-02.1, RF-02.5	● Crítica	● Baja
CU-04: Asignar piloto a bus	RF-02.6	● Crítica	● Media
CU-05: Crear alumno	RF-03.1, RF-03.7	● Crítica	● Baja
CU-06: Asignar alumno a parada	RF-03.3	● Crítica	● Media
CU-07: Crear parada	RF-04.1, RF-04.6, RF-04.7	● Crítica	● Media
CU-08: Seleccionar ubicación en mapa	RF-04.2	● Alta	● Media
CU-09: Calcular ruta automáticamente	RF-05.1, RF-05.2, RF-05.3, RF-05.4	● Crítica	● Alta
CU-10: Ver ruta en mapa	RF-05.7	● Alta	● Media
CU-11: Enviar ubicación GPS (piloto)	RF-06.1	● Crítica	● Alta
CU-12: Ver bus en tiempo real (padre)	RF-06.2, RF-06.3, RF-06.4	● Crítica	● Alta
CU-13: Registrar pago	RF-07.1, RF-07.2	● Crítica	● Media
CU-14: Confirmar pago	RF-07.5	● Crítica	● Baja

CU-15: Ver historial de pagos	RF-07.3	 Crítica	 Baja
CU-16: Confirmar asistencia	RF-08.1, RF-08.2	 Crítica	 Baja
CU-17: Registrar recogidas	RF-09.1, RF-09.2, RF-09.3, RF-09.4	 Crítica	 Media
CU-18: Ver dashboard	RF-10.1	 Alta	 Media
CU-19: Generar reporte de pagos	RF-10.2	 Alta	 Media

1.4 Análisis de Requerimientos No Funcionales

1.4.1 Requerimientos de Rendimiento (Performance)

ID	Requerimiento	Métrica	Prioridad	Justificación
RNF-01.1	El tiempo de carga de la página principal no debe exceder 2 segundos	< 2s	 Alta	Experiencia de usuario fluida
RNF-01.2	El cálculo de una ruta con hasta 30 paradas no debe tomar más de 5 segundos	< 5s	 Alta	Evitar frustración del administrador
RNF-01.3	El sistema debe soportar hasta 100 usuarios concurrentes sin degradación	100 usuarios	 Alta	Empresas medianas con muchos padres
RNF-01.4	La actualización de ubicación GPS debe ocurrir cada 10 segundos máximo	≤ 10s	 Crítica	Geolocalización en "tiempo real"
RNF-01.5	Las consultas a base de datos deben responder	< 500ms	 Alta	Fluidez en operaciones CRUD

	en menos de 500ms			
RNF-01.6	El sistema debe soportar hasta 50 buses transmitiendo ubicación simultáneamente	50 buses	● Media	Escalabilidad para empresas grandes

1.4.2 Requerimientos de Usabilidad

ID	Requerimiento	Criterio de Aceptación	Prioridad	Target
RNF-02.1	La interfaz debe ser responsive y funcionar en móviles, tablets y desktops	Bootstrap 5 responsive	● Crítica	Padres usan móviles
RNF-02.2	El sistema debe ser intuitivo: un usuario promedio debe poder usarlo sin capacitación extensa	Usuario completa tarea en < 5 min	● Alta	Pilotos/monitores no son expertos en tecnología
RNF-02.3	Los mensajes de error deben ser claros y en español	Mensajes descriptivos (no códigos técnicos)	● Alta	Evitar confusión
RNF-02.4	Los botones y enlaces deben tener tamaños adecuados para touch (mínimo 44x44px)	Guías de accesibilidad móvil	● Alta	Usabilidad en tablets para monitores
RNF-02.5	El sistema debe mostrar loaders/spinner s mientras procesa operaciones largas	Indicador visual de carga	● Alta	Feedback al usuario
RNF-02.6	Los formularios deben tener validaciones en	Validaciones client-side con JavaScript	● Media	Mejor UX

	tiempo real			
--	-------------	--	--	--

1.4.3 Requerimientos de Seguridad

ID	Requerimiento	Método de Implementación	Prioridad	Riesgo Mitigado
RNF-03.1	Todas las contraseñas deben ser hasheadas (nunca plain text)	ASP.NET Core Identity (bcrypt/PBKDF2)	● Crítica	Robo de credenciales
RNF-03.2	Las conexiones deben ser HTTPS (certificado SSL)	SSL/TLS en Azure	● Crítica	Interceptación de datos (MITM)
RNF-03.3	Las sesiones deben expirar después de 30 minutos de inactividad	Configuración de cookies de sesión	● Alta	Sesiones abiertas en computadoras compartidas
RNF-03.4	Los archivos subidos (comprobantes) deben validarse (tipo, tamaño)	Validación: JPEG/PNG, máx 5MB	● Alta	Subida de archivos maliciosos
RNF-03.5	El sistema debe prevenir SQL Injection	Entity Framework Core (ORM)	● Crítica	Ataques de inyección
RNF-03.6	El sistema debe prevenir XSS (Cross-Site Scripting)	Razor Pages sanitiza inputs automáticamente	● Crítica	Inyección de scripts maliciosos
RNF-03.7	El acceso a funcionalidades debe estar restringido por rol	[Authorize(Roles = "...")] en cada página	● Crítica	Acceso no autorizado
RNF-03.8	Los tokens de API (Google Maps) deben estar en variables de entorno, no hardcodeados	Azure Key Vault o appsettings.json no versionado	● Alta	Exposición de secretos

1.4.4 Requerimientos de Disponibilidad (Availability)

ID	Requerimiento	Métrica	Prioridad	SLA
RNF-04.1	El sistema debe estar disponible 99.5% del tiempo mensual	Uptime $\geq$ 99.5%	● Alta	~3.6 horas de downtime permitido/mes
RNF-04.2	Las actualizaciones de mantenimiento deben hacerse en horarios de baja actividad	Domingos 2-5 AM	● Media	Minimizar impacto en usuarios
RNF-04.3	El sistema debe tener backups automáticos diarios de la base de datos	Backup diario a las 3 AM	● Crítica	Recuperación ante desastres
RNF-04.4	Los backups deben retenerse por al menos 30 días	Retention: 30 días	● Alta	Recuperación de datos históricos
RNF-04.5	En caso de caída, el sistema debe recuperarse en menos de 15 minutos	RTO < 15 min	● Media	Minimizar interrupción del servicio

1.4.5 Requerimientos de Escalabilidad

ID	Requerimiento	Criterio	Prioridad	Justificación
RNF-05.1	El sistema debe soportar crecimiento de hasta 500 alumnos por empresa	500 alumnos	● Alta	Empresas grandes
RNF-05.2	La base de datos debe poder almacenar al menos 1 año de historial de ubicaciones	365 días de registros GPS	● Media	Análisis histórico
RNF-05.3	La arquitectura debe permitir agregar nuevos módulos sin refactoring mayor	Arquitectura modular (áreas Razor Pages)	● Alta	Extensibilidad futura
RNF-05.4	El sistema debe	Diseño con IdEmpresa en	● Baja	Escalabilidad del

	permitir multi-tenancy (múltiples empresas en el futuro)	tablas	(v2.0)	negocio
--	--	--------	--------	---------

1.4.6 Requerimientos de Mantenibilidad

ID	Requerimiento	Método	Prioridad	Beneficio
RNF-06.1	El código debe seguir convenciones de C# y Razor Pages	Estándares de Microsoft	● Alta	Legibilidad y mantenimiento
RNF-06.2	Las funciones complejas deben estar documentadas con comentarios XML	/// <summary> en métodos públicos	● Media	Facilitar onboarding de nuevos devs
RNF-06.3	El sistema debe tener logging de errores centralizado	Serilog o Azure Application Insights	● Alta	Debugging y monitoreo
RNF-06.4	Los cambios en base de datos deben gestionarse con migraciones de EF Core	dotnet ef migrations add	● Crítica	Control de versiones de BD
RNF-06.5	El sistema debe tener unit tests para la lógica crítica (TSP, validaciones)	xUnit, mínimo 60% code coverage	● Media	Prevenir regresiones

1.4.7 Requerimientos de Compatibilidad

ID	Requerimiento	Detalle	Prioridad
RNF-07.1	El sistema debe funcionar en navegadores modernos: Chrome, Firefox, Edge, Safari	Últimas 2 versiones de cada navegador	● Crítica
RNF-07.2	El sistema debe funcionar en dispositivos Android e iOS (vía navegador)	Responsive web, no app nativa	● Crítica
RNF-07.3	El sistema debe ser	Bootstrap	● Alta

	compatible con resoluciones desde 320px (móviles) hasta 1920px (desktops)	breakpoints	
RNF-07.4	El sistema debe funcionar con conexiones de internet moderadas (3G mínimo)	Optimización de recursos, caching	● Media

1.4.8 Requerimientos de Cumplimiento Legal

ID	Requerimiento	Normativa	Prioridad	Acción Requerida
RNF-08.1	El sistema debe cumplir con leyes de protección de datos personales (si aplican en Honduras)	GDPR (referencia), ley local	● Alta	Política de privacidad, consentimiento de padres
RNF-08.2	Los datos de menores (alumnos) deben manejarse con protección especial	COPPA (referencia)	● Alta	No recopilar datos innecesarios de alumnos
RNF-08.3	El sistema debe permitir a los usuarios exportar o eliminar sus datos (derecho al olvido)	GDPR Art. 17	● Media	Funcionalidad “Exportar mis datos” / “Eliminar mi cuenta”
RNF-08.4	El sistema debe tener términos y condiciones y política de privacidad	Legal	● Alta	Páginas estáticas con documentos legales

1.5 Análisis de Dominio

1.5.1 Modelo Conceptual del Negocio

[DIAGRAMA DE DOMINIO]

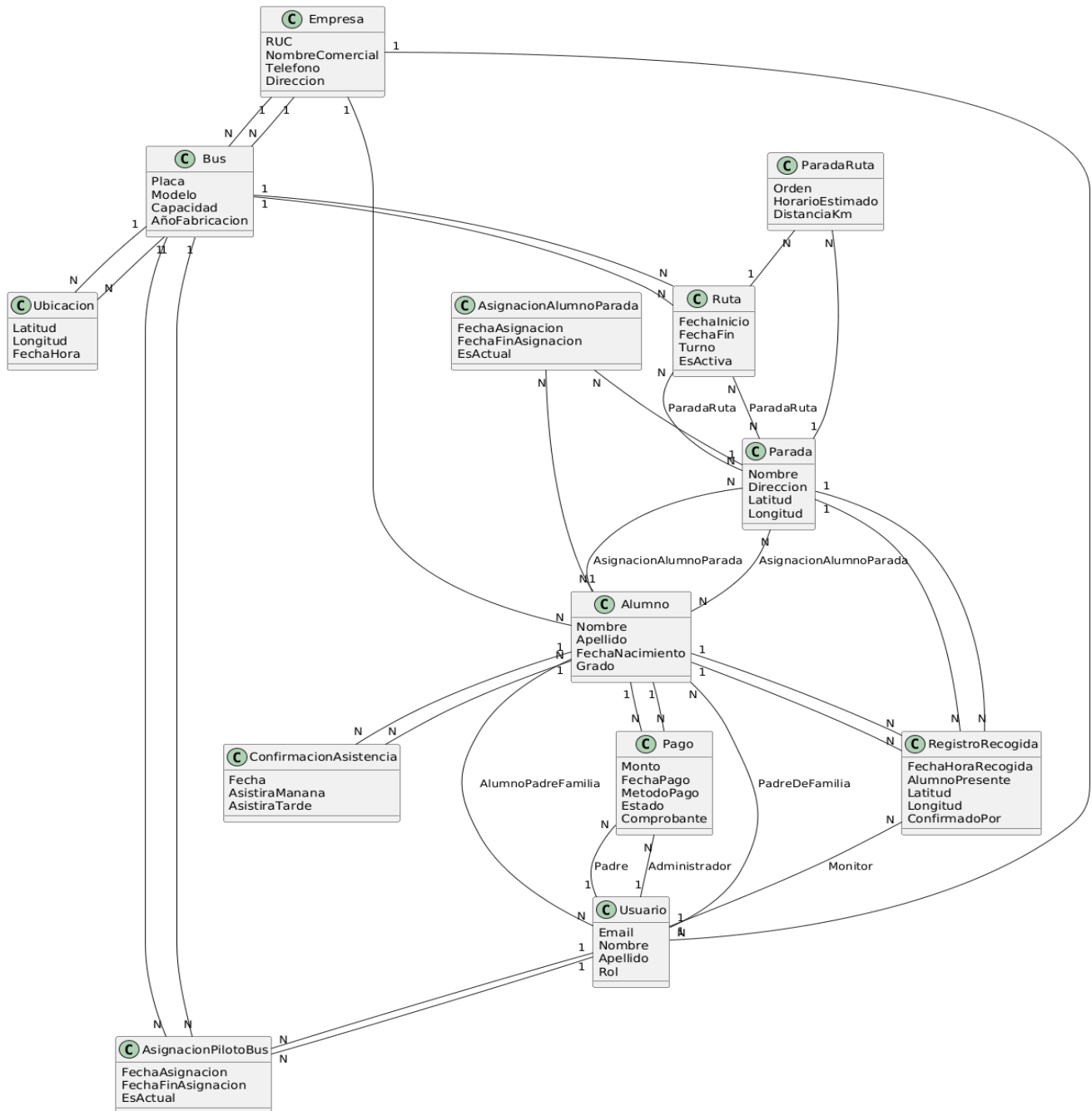
Título: Modelo de Dominio - Transportes Genesis

Entidades Principales y Relaciones:

1. Empresa (entidad raíz)
 - Atributos: RUC, NombreComercial, Teléfono, Dirección
 - Relaciones:
 - * Tiene muchos Buses (1:N)
 - * Tiene muchos Usuarios (1:N)
 - * Tiene muchos Alumnos (1:N)
2. Bus
 - Atributos: Placa (único), Modelo, Capacidad, AñoFabricación
 - Relaciones:
 - * Pertenece a una Empresa (N:1)
 - * Tiene AsignacionesPilotoBus (1:N)
 - * Tiene Rutas (1:N por turno)
 - * Transmite Ubicaciones (1:N)
3. Usuario (AppUser)
 - Atributos: Email, Nombre, Apellido, Rol (Administrador, Piloto, Monitor, PadreDeFamilia)
 - Relaciones:
 - * Si es Piloto/Monitor → Tiene AsignacionesPilotoBus (1:N)
 - * Si es Padre → Tiene Alumnos (1:N)
4. Alumno
 - Atributos: Nombre, Apellido, FechaNacimiento, Grado
 - Relaciones:
 - * Pertenece a Padres de Familia (N:N) vía AlumnoPadreFamilia
 - * Asignado a Paradas (N:N) vía AsignacionAlumnoParada
 - * Tiene ConfirmacionesAsistencia (1:N)
 - * Tiene RegistrosRecogida (1:N)
 - * Tiene Pagos asociados (1:N)
5. Parada
 - Atributos: Nombre, Dirección, Latitud, Longitud
 - Relaciones:
 - * Pertenece a Rutas (N:N) vía ParadaRuta
 - * Tiene Alumnos asignados (N:N) vía AsignacionAlumnoParada
 - * Tiene RegistrosRecogida (1:N)
6. Ruta
 - Atributos: FechaInicio, FechaFin, Turno (Mañana/Tarde), EsActiva

- Relaciones:
 - * Asignada a un Bus (N:1)
 - * Tiene Paradas (N:N) vía ParadaRuta con atributo Orden
7. ParadaRuta (tabla intermedia con datos)
 - Atributos: Orden, HorarioEstimado, DistanciaKm
 - Relaciona: Ruta ↔ Parada
 8. AsignacionPilotoBus (tabla de relación histórica)
 - Atributos: FechaAsignacion, FechaFinAsignacion, EsActual
 - Relaciona: Usuario (Piloto/Monitor) ↔ Bus
 9. AsignacionAlumnoParada (tabla de relación histórica)
 - Atributos: FechaAsignacion, FechaFinAsignacion, EsActual
 - Relaciona: Alumno ↔ Parada
 10. ConfirmacionAsistencia
 - Atributos: Fecha, AsistiraManana, AsistiraTarde
 - Relaciona: Alumno + Fecha (confirmación diaria)
 11. RegistroRecogida
 - Atributos: FechaHoraRecogida, AlumnoPresente, Latitud, Longitud, ConfirmadoPor
 - Relaciona: Alumno + Parada + Fecha + Usuario (Monitor)
 12. Pago
 - Atributos: Monto, FechaPago, MetodoPago, Estado, Comprobante
 - Relaciona: Alumno + Usuario (Padre) + Usuario (Administrador que confirma)
 13. Ubicacion (transmisión GPS en tiempo real)
 - Atributos: Latitud, Longitud, FechaHora
 - Relaciona: Bus (transmisión continua vía SignalR)

Modelo de Dominio - Transportes Genesis



Conceptos de Dominio (Glosario):

Turno: Periodo del día (Mañana o Tarde) en que opera una ruta

Ruta Activa: Ruta vigente y en operación

Orden de Parada: Secuencia numérica en que se visitan las paradas en una ruta

TSP (Traveling Salesman Problem): Algoritmo que calcula la ruta más corta

visitando todas las paradas

Geolocalización: Ubicación geográfica expresada en latitud y longitud

SignalR: Tecnología de comunicación en tiempo real (WebSockets)

Rol: Perfil de usuario con permisos específicos

Soft Delete: Eliminación lógica (registro marcado como eliminado pero no borrado físicamente)

1.5.2 Reglas de Negocio

ID	Regla de Negocio	Justificación	Implementación
RN-01	Un bus solo puede tener un piloto activo a la vez	Evitar confusión sobre quién opera el bus	Campo EsActual en AsignacionPilotoBus
RN-02	Un alumno solo puede estar asignado a una parada activa a la vez	Un alumno solo puede ser recogido en una ubicación por turno	Campo EsActual en AsignacionAlumnoParada
RN-03	Un bus solo puede tener una ruta activa por turno (Mañana y Tarde)	Evitar ambigüedad sobre qué ruta seguir	Campo Turno + EsActiva en Ruta
RN-04	Una confirmación de asistencia solo es válida para el día actual	Confirmaciones son diarias y no se arrastran	Campo Fecha en ConfirmacionAsistencia
RN-05	No se puede registrar la misma recogida dos veces (mismo alumno, misma parada, mismo día)	Evitar duplicados	Validación en controller/service
RN-06	Un pago en estado "Pendiente" puede ser editado por el padre; uno "Confirmado" no	Integridad de registros contables	Validación de estado antes de editar
RN-07	La fecha de nacimiento de un alumno debe estar en el rango de 3-18 años	Alumnos de transporte escolar son niños/adolescentes	Validación en modelo
RN-08	La placa de un bus debe ser única en el sistema	Identificación inequívoca de buses	Índice único en BD + validación
RN-09	Un usuario con rol "Piloto" o "Monitor"	Sin asignación, no hay ruta que mostrar	Validación en página MiRuta.cshtml.cs

	debe estar asignado a un bus para acceder a su ruta		
RN-10	El cálculo de ruta debe considerar solo paradas con alumnos asignados	No tiene sentido visitar paradas vacías	Filtro en servicio de cálculo de rutas
RN-11	Los horarios estimados de paradas se calculan asumiendo velocidad promedio de 30 km/h	Estimación realista para zonas urbanas	Fórmula: distancia / velocidad
RN-12	Un padre solo puede ver información de sus propios hijos	Privacidad y seguridad	Filtro por IdUsuarioPadre en consultas
RN-13	Al eliminar un bus (soft delete), las rutas asociadas se marcan como inactivas	Consistencia de datos	Lógica en servicio de eliminación
RN-14	La ubicación GPS debe actualizarse máximo cada 10 segundos para considerarse "tiempo real"	Balance entre precisión y consumo de datos	Timer en cliente SignalR
RN-15	Solo el administrador puede confirmar o rechazar pagos	Control financiero centralizado	[Authorize(Roles = "Administrador")]

1.5.3 Procesos de Negocio Clave

Proceso 1: Onboarding de Nuevo Alumno

[Administrador]

- ↓
1. Crea usuario "Padre de Familia" (email + contraseña temporal)
- ↓
2. Crea registro de Alumno (nombre, apellido, fecha nacimiento, grado)
- ↓
3. Vincula Alumno con Padre (tabla AlumnoPadreFamilia)
- ↓
4. Asigna Alumno a una Parada existente (tabla AsignacionAlumnoParada, EsActual = true)
- ↓
5. Recalcula rutas afectadas (servicio TSP)
- ↓

6. Notifica a Padre (por correo o WhatsApp) con credenciales de acceso

↓

[Padre recibe email, accede al sistema, cambia contraseña obligatoriamente]

Proceso 2: Operación Diaria de Ruta (Turno Mañana)

Día Anterior (Noche):

[Padre de Familia]

↓

Confirma asistencia de su hijo para mañana (página "Confirmar Asistencia")

↓

Sistema registra en `ConfirmacionAsistencia` (Fecha = mañana, AsistiraManana = true)

Día Actual (Mañana):

[Piloto]

↓

1. Inicia sesión, ve su ruta diaria en `MiRuta`

↓

2. Revisa lista de alumnos confirmados

↓

3. Inicia transmisión de ubicación GPS (activo botón "Iniciar Ruta")

↓

4. SignalR envía ubicación cada 10 segundos

[Monitor] (en paralelo)

↓

1. Abre página `RegistrarRecogidas` en tablet

↓

2. Ve ruta con paradas y alumnos

↓

3. En cada parada:

- Marca checkboxes de alumnos que subieron al bus

- Presiona "Guardar"

- Sistema registra en `RegistroRecogida` (FechaHora, Lat, Lon automáticos)

↓

4. Continúa hasta completar todas las paradas

[Padre de Familia] (en paralelo)

↓

1. Abre el sistema desde su móvil

↓

2. Ve mapa con bus en tiempo real

↓

3. Observa cuándo el bus está cerca de su parada
↓
4. Confirma que su hijo fue recogido (ve checkbox marcado en su panel)

[Administrador]

↓

Al final del día, revisa dashboard:

- Alumnos confirmados vs recogidos
- Rutas completadas
- Incidencias (alumnos confirmados, pero no recogidos)

Proceso 3: Ciclo de Pago Mensual

Inicio de Mes:

[Administrador]

↓

Genera factura/recibo mensual para cada alumno (puede ser manual o futura feature)

↓

Envía por email o WhatsApp a padres

Durante el Mes:

[Padre de Familia]

↓

1. Hace depósito bancario o transferencia

↓

2. Accede al sistema, sección "Mis Pagos"

↓

3. Clic en "Registrar Pago"

↓

4. Completo formulario:

- Alumno
- Monto
- Fecha de pago
- Método (Transferencia/Efectivo/Tarjeta)
- Sube imagen del comprobante

↓

5. Presiona "Guardar"

↓

Sistema guarda pago con Estado = "Pendiente"

Validación:

[Administrador]

↓

1. Ve listado de "Pagos Pendientes"

↓

2. Revisa comprobante subido

↓

3. Valida contra registros bancarios

↓

4. Opciones:

a) Confirmar pago → Estado = "Confirmado"

b) Rechazar pago → Estado = "Rechazado", ingresa motivo ("Monto incorrecto", "Comprobante ilegible", etc.)

↓

Sistema actualiza estado

Consulta:

[Padre de Familia]

↓

Entra a "Historial de Pagos"

↓

Ve tabla con todos sus pagos:

- Fecha
- Monto
- Estado (Pendiente / Confirmado / Rechazado)
- Motivo (si fue rechazado)

1.5.4 Eventos de Dominio

Evento	Desencadenante	Efecto en el Sistema	Actores Notificados
AlumnoAsignadoAParada	Administrador asigna alumno a parada	Recalculo automático de ruta del bus correspondiente	Piloto, Monitor (ven ruta actualizada)
RutaCalculada	Administrador ejecuta "Calcular Ruta"	Se genera Ruta con ParadaRuta ordenadas	Piloto, Monitor
UbicacionActualizada	Piloto envía GPS cada 10s	Mapa de padres se actualiza en tiempo real	Padres de alumnos en esa ruta
AsistenciaConfirmada	Padre confirma asistencia	Piloto/Monitor ven lista actualizada de confirmaciones	Piloto, Monitor
RecogidaRegistrada	Monitor marca alumno como recogido	Padre ve notificación visual de que su hijo subió al bus	Padre de ese alumno
PagoRegistrado	Padre sube pago	Aparece en lista de pagos pendientes del administrador	Administrador
PagoConfirmado	Administrador confirma pago	Estado del pago cambia a "Confirmado"	Padre ve confirmación en su historial

PilotoAsignadoABus	Administrador asigna piloto	Piloto puede acceder a su ruta	Piloto
---------------------------	-----------------------------	--------------------------------	--------

1.6 Supuestos, restricciones y dependencias funcionales

1.6.1 Supuestos del Proyecto

ID	Supuesto	Implicación	Riesgo si es Falso
SUP-01	Los usuarios (pilotos, monitores, padres) tienen acceso a smartphones o computadoras con navegador web	El sistema es 100% web; no hay app nativa	● Medio: Si muchos usuarios no tienen dispositivos, adopción será baja
SUP-02	Los usuarios tienen conexión a internet (3G mínimo) durante la operación	Geolocalización y actualización en tiempo real dependen de conectividad	● Alto: Sin internet, features críticas no funcionan
SUP-03	Los navegadores de los usuarios son modernos (últimas 2 versiones de Chrome, Firefox, Edge, Safari)	Se usan features modernas de HTML5, CSS3, JavaScript	● Bajo: Navegadores antiguos son minoría
SUP-04	La empresa de transporte ya tiene definidas sus paradas y rutas operativas	El sistema digitaliza rutas existentes	● Medio: Si no hay rutas establecidas, requiere trabajo previo de campo
SUP-05	Los pilotos y monitores tienen dispositivos (tablets o móviles) para usar durante la operación	Registro de recogidas y envío de GPS ocurren en movimiento	● Medio: Si no tienen dispositivos, la empresa debe proveerlos
SUP-06	Los padres están dispuestos a usar una plataforma digital	Cambio cultural de WhatsApp/llamadas a	● Medio: Resistencia al cambio puede frenar adopción

		sistema web	
SUP-07	Google Maps API está disponible y accesible en Honduras	Geolocalización y mapas dependen de Google	● Bajo: Google Maps funciona globalmente
SUP-08	La empresa tiene capacidad de invertir en infraestructura cloud (Azure)	Costos operativos de \$150-\$300/mes	● Medio: Si no puede pagar, el sistema no puede operarse
SUP-09	Los datos de ubicación GPS de los dispositivos son precisos (margen de error ≤ 10 metros)	Mapas en tiempo real muestran posición real del bus	● Bajo: GPS moderno es generalmente preciso
SUP-10	La empresa tiene personal (administrador) con conocimientos básicos de computación	Gestión del sistema requiere uso de interfaces CRUD	● Medio: Si el administrador no sabe usar computadoras, requiere capacitación intensiva

1.6.2 Restricciones del Proyecto

Restricciones Técnicas

ID	Restricción	Tipo	Impacto
RES-TEC-01	El sistema DEBE desarrollarse en .NET 8 con Razor Pages	Tecnología	No se puede usar Blazor, Angular, React, etc.
RES-TEC-02	La base de datos DEBE ser SQL Server	Infraestructura	No se puede usar MySQL, PostgreSQL, MongoDB, etc.
RES-TEC-03	El sistema DEBE desplegarse en Microsoft Azure	Infraestructura	No se puede usar AWS, Google Cloud, DigitalOcean, etc.
RES-TEC-04	La geolocalización DEBE usar Google Maps API	API Externa	No se puede usar OpenStreetMap, Mapbox (al menos en v1.0)
RES-TEC-05	El tiempo real DEBE	Protocolo	No se puede usar

	implementarse con SignalR		polling, long-polling, otros frameworks WebSocket
RES-TEC-06	El sistema DEBE ser una aplicación web (NO app móvil nativa)	Plataforma	No hay versiones iOS/Android nativas en v1.0

Restricciones de Presupuesto

ID	Restricción	Límite	Impacto
RES-PRE-01	Inversión inicial máxima: Q16,000 USD	Desarrollo + capital de trabajo	No se pueden contratar más de 2-3 desarrolladores o extender timeline excesivamente
RES-PRE-02	Costos operativos mensuales máximos: Q2000 USD	Azure + Google Maps + otros	Debe optimizarse el uso de recursos cloud
RES-PRE-03	Precio de venta por cliente: Q 60/bus/mes	Modelo de negocio	No se puede cobrar más sin justificación de valor

Restricciones de Tiempo

ID	Restricción	Plazo	Impacto
RES-TIE-01	El MVP debe estar listo en 3 meses (12 semanas)	537 horas de desarrollo	Features no críticas deben diferirse a v2.0
RES-TIE-02	El sistema debe estar operativo para el inicio del ciclo escolar	Típicamente Febrero en Honduras	Deadline fijo; retrasos pueden perder oportunidad de mercado

Restricciones Legales y de Cumplimiento

ID	Restricción	Normativa	Impacto
RES-LEG-01	El sistema debe cumplir con leyes de protección de datos personales	Ley hondureña (si existe) o GDPR como referencia	Debe incluirse política de privacidad y consentimiento
RES-LEG-02	Los datos de menores (alumnos) deben manejarse con protección especial	COPPA (referencia)	No recopilar datos sensibles innecesarios
RES-	El sistema debe tener	Legal	Página estática

LEG-03	términos y condiciones		obligatoria
---------------	------------------------	--	-------------

1.6.3 Dependencias Funcionales

Dependencias Externas (Third-Party)

Dependencia	Proveedor	Criticidad	Función	Riesgo de Disponibilidad	Mitigación
Google Maps API	Google	● Crítica	Geolocalización, mapas, cálculo de distancias	● Baja (99.9% uptime)	Cache de mapas estáticos, fallback a OpenStreetMap en v2.0
Microsoft Azure	Microsoft	● Crítica	Hosting, base de datos, storage	● Baja (SLA 99.95%)	Backups regulares, plan de recuperación ante desastres
SignalR	Microsoft	● Crítica	Tiempo real (WebSockets)	● Baja (parte de .NET)	Fallback a long-polling si WebSockets falla
Bootstrap 5	Open Source (CDN)	● Alta	UI responsive	● Baja (múltiples CDNs)	Servir Bootstrap desde servidor propio si CDN falla
jQuery	Open Source (CDN)	● Alta	Interactividad frontend	● Baja	Servir desde servidor propio

Dependencias Internas (Entre Módulos)

Módulo Dependiente	Módulo del que Depende	Tipo de Dependencia	Impacto si el Módulo Base Falla
Cálculo de Rutas	Gestión de Paradas + Gestión de Alumnos	Datos	No se puede calcular ruta sin paradas y alumnos
Geolocalización en Tiempo Real	Gestión de Buses + Gestión de Rutas	Datos	No se puede mostrar bus en mapa sin bus y ruta asignados

Registro de Recogidas	Gestión de Rutas + Gestión de Alumnos	Datos	Monitor no puede registrar recogidas sin ruta y alumnos
Confirmación de Asistencia	Gestión de Alumnos + Auth (Padres)	Datos + Auth	Padre no puede confirmar sin alumno vinculado
Gestión de Pagos	Gestión de Alumnos + Auth (Padres, Administrador)	Datos + Auth	No se puede registrar pago sin alumno y usuarios
Dashboard de Administrador	Todos los módulos	Datos agregados	Dashboard completo y sistema sigue operativo

Dependencias de Datos (Secuencia de Implementación)

Orden obligatorio de configuración inicial:

1. Crear Empresa
↓
2. Crear Usuarios (Administrador, Pilotos, Monitores, Padres)
↓
3. Crear Buses
↓
4. Asignar Pilotos a Buses (AsignacionPilotoBus)
↓
5. Crear Paradas (con geolocalización)
↓
6. Crear Alumnos
↓
7. Vincular Alumnos con Padres (AlumnoPadreFamilia)
↓
8. Asignar Alumnos a Paradas (AsignacionAlumnoParada)
↓
9. Calcular Rutas (TSP automático)
↓
10. Iniciar Operación Diaria (confirmaciones, registro de recogidas, geolocalización)

Implicación: No se puede “saltar” pasos; cada módulo depende del anterior.

1.6.4 Riesgos Funcionales

ID	Riesgo	Probabilidad	Impacto	Mitigación
RF-RIESGO-01	El algoritmo TSP no optimiza rutas adecuadamente para casos complejos (50+ paradas)	● Media	● Medio	Implementar versión simplificada (greedy algorithm) + permitir ajuste manual
RF-RIESGO-02	Los pilotos olvidan activar la transmisión de ubicación GPS	● Media	● Alto	Mostrar alerta prominente en su dashboard + notificación si no transmite en horario de ruta
RF-RIESGO-03	Los monitores registran recogidas incorrectamente (marcan alumnos que no subieron)	● Media	● Medio	Validación cruzada con confirmaciones de asistencia + revisión de administrador
RF-RIESGO-04	Los padres no confirman asistencia (sistema asume que todos asistirán)	● Alta	● Bajo	Configuración: "Por defecto, todos asisten si no se confirma lo contrario"
RF-RIESGO-05	La conexión a internet del bus es intermitente, ubicación GPS se actualiza con retraso	● Media	● Medio	Buffer de ubicaciones en cliente, enviar cuando reconecta + mostrar "última ubicación conocida" en mapa
RF-RIESGO-06	Los padres suben comprobantes de pago falsos o editados	● Baja	● Alto	Validación manual por administrador + verificación cruzada con extractos bancarios
RF-RIESGO-07	El sistema se usa para un solo bus (no escala a múltiples buses)	● Baja	● Bajo	Arquitectura ya soporta múltiples buses; solo es cuestión de configuración

1.1.5 Usuarios Finales del Sistema

El sistema está diseñado para cuatro tipos principales de usuarios:

1. **Administradores** (Gerentes/Dueños de la empresa)
 - Gestionan usuarios, buses, rutas, alumnos
 - Calculan rutas optimizadas
 - Visualizan reportes y estadísticas
 - Supervisan operaciones en tiempo real
2. **Pilotos** (Conductores de buses)
 - Visualizan su ruta asignada del día
 - Ven la secuencia de paradas y horarios
 - Consultan información de alumnos en cada parada
 - Acceden al mapa de su ruta
3. **Monitores** (Asistentes en el bus)
 - Visualizan la ruta del bus asignado
 - Registran qué alumnos fueron recogidos en cada parada
 - Confirman presencia o ausencia de alumnos
 - Acceden al mapa en tiempo real
4. **Padres de Familia** (Clientes)
 - Confirman asistencia diaria de sus hijos
 - Visualizan el bus en tiempo real
 - Registran pagos realizados
 - Consultan historial de pagos y asistencias

1.1.6 Beneficios Esperados

Para la Empresa de Transporte:

- **Reducción de costos** operativos (combustible) mediante rutas optimizadas
- **Mejora en eficiencia** operativa (procesos automatizados)
- **Mayor control** y trazabilidad de operaciones
- **Reducción de errores** humanos en registros
- **Mejor imagen corporativa** (tecnología moderna)
- **Escalabilidad** del negocio (más fácil gestionar más buses)

Para los Pilotos y Monitores:

- **Claridad en rutas** a seguir cada día
- **Reducción de tiempo** en planificación manual
- **Facilidad de registro** de recogidas (digital vs papel)
- **Acceso desde cualquier dispositivo** con internet

Para los Padres de Familia:

- **Tranquilidad** al ver ubicación del bus en tiempo real

- **Mejor comunicación** con la empresa (confirmaciones, pagos)
- **Transparencia** en registros y pagos
- **Facilidad de uso** (interfaz web responsive)

ESPECIFICACIÓN TÉCNICA - TRANSPORTES GENESIS

2.1 Arquitectura de referencia

2.1.1 Diagrama C4 - Nivel 1: Contexto del Sistema

[INSTRUCCIONES PARA DIAGRAMA]

Diagrama de Contexto - Transportes Genesis

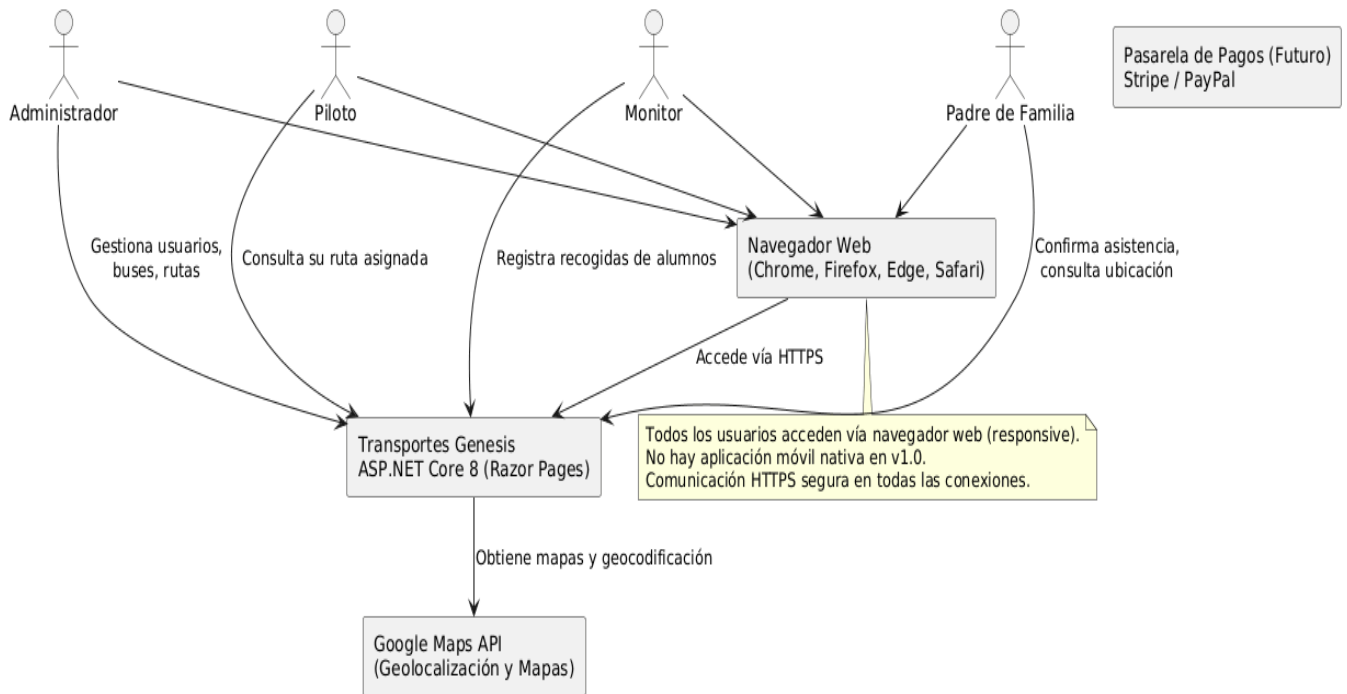
Elementos:

1. Sistema Central: "Transportes Genesis"
 - Tipo: Sistema de Software
 - Tecnología: ASP.NET Core 8 (Razor Pages)
2. Actores Externos (Personas):
 - Administrador (Gerente/Dueño)
 - Piloto (Conductor)
 - Monitor (Asistente del bus)
 - Padre de Familia (Cliente)
3. Sistemas Externos:
 - Google Maps API (Geolocalización y mapas)
 - Navegador Web (Chrome, Firefox, Edge, Safari)
 - [Futuro] Pasarela de Pagos (Stripe, PayPal)

Relaciones:

- Administrador → Transportes Genesis: "Gestiona usuarios, buses, rutas"
- Piloto → Transportes Genesis: "Consulta su ruta asignada"
- Monitor → Transportes Genesis: "Registra recogidas de alumnos"
- Padre → Transportes Genesis: "Confirma asistencia, consulta ubicación"
- Transportes Genesis → Google Maps API: "Obtiene mapas y geocodificación"
- Usuarios → Navegador Web → Transportes Genesis: "Accede vía HTTPS"

Diagrama de Contexto - Transportes Genesis



2.1.2 Diagrama C4 - Nivel 2: Contenedores

Diagrama de Contenedores - Transportes Genesis

Contenedores:

1. "Web Application" (ASP.NET Core Razor Pages)
 - Tecnología: .NET 8, Razor Pages, MVC Controllers
 - Puerto: 443 (HTTPS)
 - Responsabilidad: Interfaz de usuario, lógica de presentación
 - Componentes: Pages/, Controllers/, Views/
2. "API REST" (ASP.NET Core Web API)
 - Tecnología: .NET 8, ASP.NET Core Controllers
 - Responsabilidad: Endpoints para operaciones CRUD, cálculo de rutas
 - Ruta base: /api/
3. "SignalR Hub" (Tiempo Real)
 - Tecnología: SignalR for .NET 8
 - Responsabilidad: Comunicación bidireccional en tiempo real
 - Uso: Geolocalización de buses en vivo

4. "Base de Datos" (SQL Server)

- Tecnología: SQL Server 2019+
- Puerto: 1433
- Responsabilidad: Almacenamiento persistente de datos
- Esquemas: genesis (negocio), dbo (Identity)

5. "Cliente Web" (Navegador)

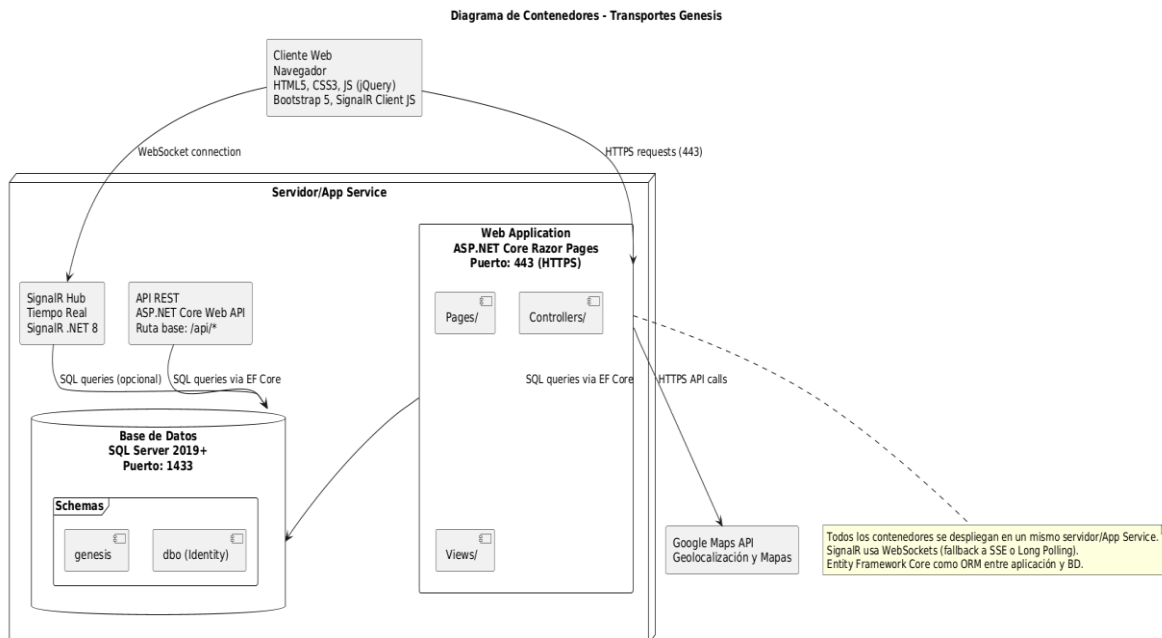
- Tecnología: HTML5, CSS3, JavaScript (jQuery)
- Frameworks: Bootstrap 5, SignalR Client JS
- Responsabilidad: Renderizado de UI, interacción con usuario

Relaciones:

- Cliente Web → Web Application: "HTTPS requests" (puerto 443)
- Cliente Web → SignalR Hub: "WebSocket connection"
- Web Application → Base de Datos: "SQL queries via EF Core"
- API REST → Base de Datos: "SQL queries via EF Core"
- SignalR Hub → Base de Datos: "SQL queries (opcional)"
- Web Application → Google Maps API: "HTTPS API calls"

Notas:

- Todos los contenedores se despliegan en un mismo servidor/App Service
- SignalR usa WebSockets (fallback a Server-Sent Events o Long Polling)
- Entity Framework Core como ORM entre aplicación y BD



2.1.3 Diagrama C4 - Nivel 3: Componentes (Web Application)

Diagrama de Componentes - Web Application

Componentes principales:

1. "Authentication & Authorization" (ASP.NET Core Identity)
 - Archivos: Controllers/AuthController.cs, Startup.cs
 - Responsabilidad: Login, logout, gestión de sesiones
 - Usa: AspNetUsers, AspNetRoles, AspNetUserRoles
2. "Pages - Admin" (Razor Pages)
 - Carpeta: Pages/Admin/
 - Páginas: Index.cshtml, CalcularRutas.cshtml, CrearUsuariosPrueba.cshtml
 - Responsabilidad: Panel de administración
3. "Pages - Piloto" (Razor Pages)
 - Carpeta: Pages/Piloto/
 - Páginas: MiRuta.cshtml
 - Responsabilidad: Vista de ruta para pilotos
4. "Pages - Monitor" (Razor Pages)
 - Carpeta: Pages/Monitor/
 - Páginas: MiRuta.cshtml, RegistrarRecogidas.cshtml
 - Responsabilidad: Vista de ruta y registro de recogidas
5. "Pages - Padres" (Razor Pages)
 - Carpeta: Pages/Padres/
 - Páginas: ConfirmarAsistencia.cshtml
 - Responsabilidad: Confirmación de asistencia de alumnos
6. "Pages - Geolocalización" (Razor Pages)
 - Carpeta: Pages/Geolocalizacion/
 - Páginas: MapaEnTiempoReal.cshtml
 - Responsabilidad: Mapa en vivo con ubicación de buses

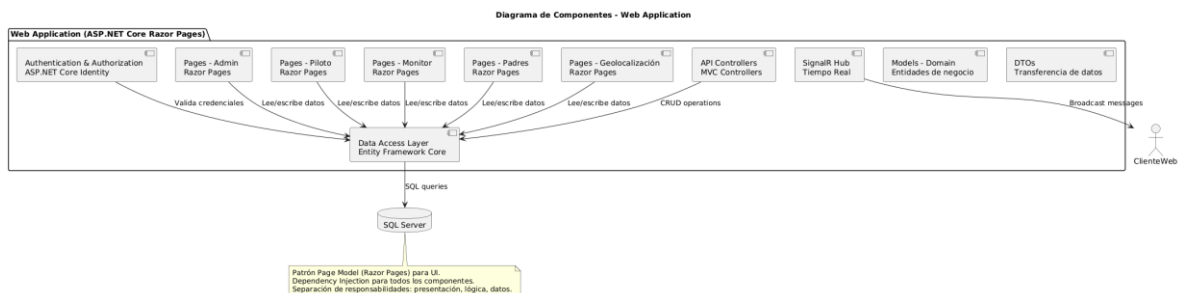
7. "API Controllers" (MVC Controllers)
 - Archivos: Controllers/RutasController.cs, Controllers/PagosController.cs
 - Responsabilidad: Endpoints REST para operaciones
8. "SignalR Hub"
 - Archivos: Hubs/GeolocalizacionHub.cs
 - Responsabilidad: Broadcast de ubicaciones de buses
9. "Data Access Layer" (Entity Framework Core)
 - Archivos: Data/Context/ApplicationDbContext.cs
 - Responsabilidad: Acceso a base de datos, DbSet
10. "Models - Domain"
 - Carpeta: Models/DB/Negocio/, Models/DB/Usuarios/
 - Archivos: Bus.cs, Ruta.cs, Parada.cs, Alumnos.cs, etc.
 - Responsabilidad: Entidades de dominio
11. "DTOs"
 - Carpeta: DTOs/
 - Archivos: RutaDto.cs, PagoDto.cs, etc.
 - Responsabilidad: Transferencia de datos entre capas

Relaciones:

- Authentication → Data Access Layer: "Valida credenciales"
- Pages → Data Access Layer: "Lee/escribe datos"
- API Controllers → Data Access Layer: "CRUD operations"
- SignalR Hub → Clientes: "Broadcast messages"
- Data Access Layer → SQL Server: "SQL queries"

Notas:

- Patrón Page Model (Razor Pages) para UI
- Dependency Injection para todos los componentes
- Separation of Concerns (presentación, lógica, datos)



2.1.4 Estilo Arquitectónico y Justificación

Estilo Arquitectónico Principal

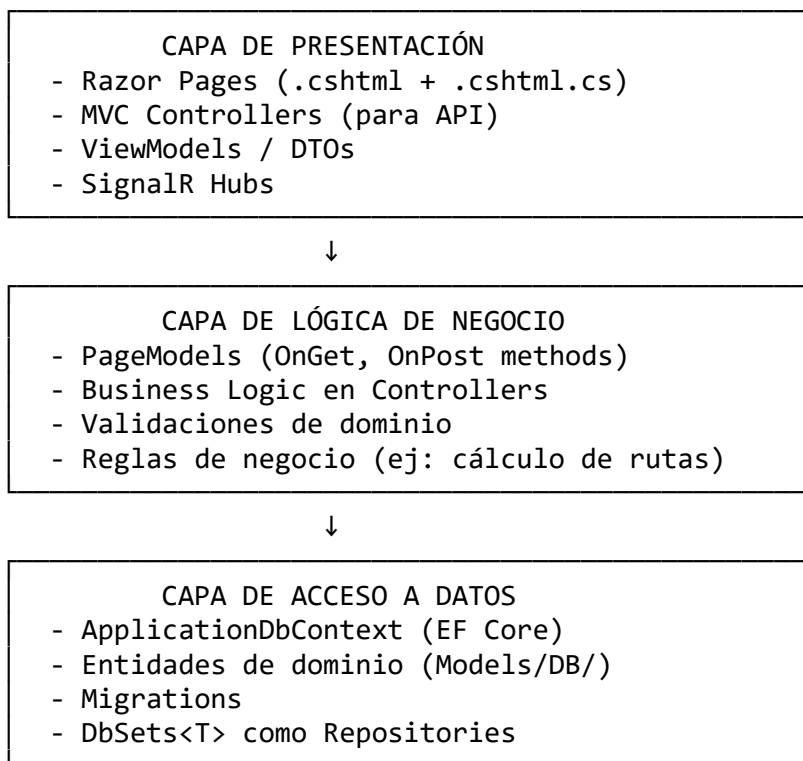
Razor Pages con Patrón Page-Focused MVC

El proyecto está construido principalmente con Razor Pages, un patrón de diseño de ASP.NET Core que simplifica el desarrollo de aplicaciones web orientadas a páginas. Este estilo se complementa con:

- MVC Controllers para endpoints API REST
- SignalR Hub para comunicación en tiempo real
- Repository Pattern implícito mediante Entity Framework Core

Capas de la Aplicación

La arquitectura sigue una separación de responsabilidades en 3 capas lógicas:





BASE DE DATOS
- SQL Server 2019+
- Esquema 'genesis' (negocio)
- Esquema 'dbo' (ASP.NET Core Identity)

Justificación de la Arquitectura Elegida

¿Por qué Razor Pages?

1. **Productividad:** Desarrollo rápido de páginas web sin complejidad excesiva
2. **Simplicidad:** Patrón Page-Focused más intuitivo que MVC tradicional para aplicaciones CRUD
3. **Integración nativa con .NET:** Soporte oficial de Microsoft, excelente documentación
4. **Separación de concerns:** El PageModel separa lógica de presentación (CSHTML) de lógica de negocio (CS)
5. **Madurez:** Tecnología estable y probada en producción

¿Por qué complementar con MVC Controllers?

- Para endpoints API REST (/api/*) que no requieren renderizado de UI
- Separación clara entre páginas web (Razor Pages) y servicios API (Controllers)

¿Por qué SignalR?

- **Comunicación en tiempo real necesaria para geolocalización de buses**
- **Protocolo WebSocket eficiente (bajo overhead)**
- **Integración nativa con ASP.NET Core**

Alternativas consideradas:

Alternativa	Ventaja	Desventaja	Decisión
Blazor Server	UI más reactiva, C# en cliente	Mayor complejidad, estado en servidor	Rechazado (overkill para v1.0)
Blazor WebAssembly	SPA, C# en navegador	Tamaño de descarga grande, curva de aprendizaje	Rechazado (no necesario)
Angular/React + API	SPA moderna, experiencia de usuario rica	Requiere equipo frontend separado, más tiempo de desarrollo	Rechazado (fuera de alcance)
MVC tradicional	Patrón conocido	Más verboso que Razor Pages	Usado solo para API

Razor Pages + MVC + SignalR	Balance perfecto, productividad alta, tiempo real	Ninguna significativa	ELEGIDO
------------------------------------	---	-----------------------	---------

2.1.5 Diagrama de Despliegue

[INSTRUCCIONES PARA DIAGRAMA UML]

Título: Diagrama de Despliegue - Transportes Genesis

Nodos:

1. "Cliente" (Device)
 - Tipo: Dispositivo de usuario (PC, Tablet, Smartphone)
 - Software: Navegador Web (Chrome, Firefox, Edge, Safari)
 - Componentes:
 - HTML5/CSS3/JavaScript
 - Bootstrap 5 (UI)
 - SignalR Client (JS)
2. "Servidor Web / App Service" (Execution Environment)
 - Tipo: IIS 10 / Azure App Service
 - Sistema Operativo: Windows Server 2019+ / Azure
 - Runtime: .NET 8 Runtime
 - Componentes:
 - ASP.NET Core Application (Razor Pages + API)
 - SignalR Hub
 - Kestrel Web Server (detrás de IIS)
3. "Servidor de Base de Datos" (Database Server)
 - Tipo: SQL Server 2019+
 - Sistema Operativo: Windows Server / Azure SQL Database
 - Puerto: 1433 (TCP)
 - Componentes:
 - SQL Server Database Engine
 - Base de datos: TransportesGenesisDb
 - Esquemas: genesis, dbo
4. "Servicios Externos" (External System)
 - Google Maps API (HTTPS, puerto 443)
 - [Futuro] Pasarela de Pagos

Conexiones:

- Cliente → Servidor Web: "HTTPS (puerto 443)"
- Protocolo: HTTP/1.1, HTTP/2

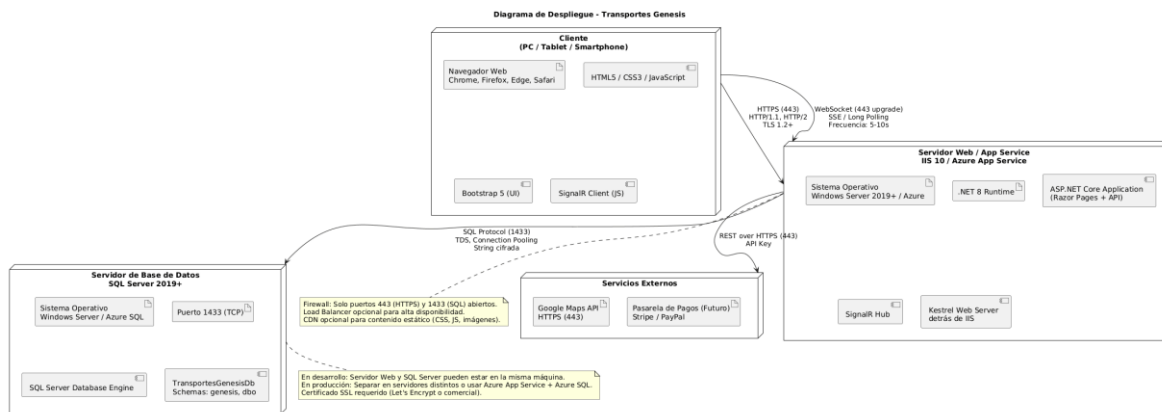
- Seguridad: TLS 1.2+
- Cliente → SignalR Hub: "WebSocket (puerto 443 upgrade)"
 - Protocolo: WebSocket (fallback: SSE, Long Polling)
 - Frecuencia: cada 5-10 segundos
- Servidor Web → SQL Server: "SQL Protocol (puerto 1433)"
 - Protocolo: TDS (Tabular Data Stream)
 - Conexión: Connection Pooling habilitado
 - String de conexión cifrada
- Servidor Web → Google Maps API: "HTTPS (puerto 443)"
 - Protocolo: REST over HTTPS
 - Autenticación: API Key

Configuración de Red:

- Firewall: Solo puertos 443 (HTTPS) y 1433 (SQL) abiertos
- Load Balancer (opcional): Para alta disponibilidad
- CDN (opcional): Para contenido estático (CSS, JS, imágenes)

Notas:

- En desarrollo: Servidor Web y SQL Server pueden estar en la misma máquina
- En producción: Separar en servidores distintos o usar Azure App Service + Azure SQL
- Certificado SSL requerido (Let's Encrypt gratuito o comercial)



2.1.6 Principios de Diseño Aplicados

El diseño arquitectónico del sistema se basa en los siguientes principios:

1. *Separation of Concerns (SoC)*

- La lógica de presentación (CSHTML) está separada de la lógica de negocio (PageModel)
- La capa de datos (ApplicationDbContext) está aislada del resto de la aplicación
- Cada componente tiene una responsabilidad única y bien definida

2. *Dependency Injection (DI)*

- Todos los servicios se registran en Startup.cs (o Program.cs en .NET 8)
- Las dependencias se inyectan vía constructor
- Facilita testing y desacoplamiento

Ejemplo:

```
public MiRutaModel(  
    IHttpConnectionFactory httpConnectionFactory,  
    ApplicationDbContext context)  
{  
    _httpConnectionFactory = httpConnectionFactory;  
    _context = context;  
}
```

3. *Don't Repeat Yourself (DRY)*

- Uso de componentes parciales (_Layout.cshtml)
- DTOs reutilizables (RutaDto, ParadaRutaDto)
- Clases base con auditoría (Auditoria.cs)

4. *SOLID Principles*

- **Single Responsibility:** Cada PageModel tiene una única responsabilidad
- **Open/Closed:** Extensible mediante herencia (ej: Auditoria base)
- **Liskov Substitution:** Clases derivadas son intercambiables
- **Interface Segregation:** Interfaces específicas (ej: IHttpConnectionFactory)
- **Dependency Inversion:** Dependencias a abstracciones, no implementaciones

5. *Convention over Configuration*

- Razor Pages usa convenciones de carpetas (Pages/)
- Entity Framework usa convenciones de nomenclatura

- Reducción de configuración manual

6. Security by Design

- Autenticación obligatoria ([Authorize])
- Autorización basada en roles ([Authorize(Roles = "...")])
- Anti-CSRF tokens en formularios
- HTTPS obligatorio
- Hashing de contraseñas con ASP.NET Core Identity

2.2 Stack tecnológico seleccionado

2.2.1 Tecnologías por Capa

Backend (Servidor)

Tecnología	Versión	Licencia	Propósito	Justificación
.NET	8.0 LTS	MIT	Plataforma de desarrollo	Última versión LTS, soporte hasta 2026, alto rendimiento, multiplataforma
ASP.NET Core	8.0	MIT	Framework web	Framework moderno, cross-platform, alto rendimiento, excelente para APIs REST y Razor Pages
Razor Pages	8.0	MIT	Framework UI	Patrón simplificado para páginas web, productividad alta, menos verboso que MVC tradicional
Entity Framework Core	8.0	MIT	ORM	Acceso a datos con LINQ, migraciones automáticas, soporte para SQL Server, reduce código boilerplate
ASP.NET Core Identity	8.0	MIT	Autenticación /Autorización	Sistema completo de gestión de usuarios, roles, claims, integración nativa con EF Core
SignalR	8.0	MIT	Comunicación en tiempo real	WebSockets para geolocalización en vivo, fallback

				automático, integración nativa con ASP.NET Core
C#	12.0	MIT	Lenguaje de programación	Lenguaje tipado, moderno, orientado a objetos, async/await nativo, LINQ

Frontend (Cliente)

Tecnología	Versión	Licencia	Propósito	Justificación
HTML5	-	W3C	Estructura de páginas	Estándar web moderno, soporte para semántica, accesibilidad
CSS3	-	W3C	Estilos y diseño	Flexbox, Grid, animaciones, variables CSS
Bootstrap	5.3	MIT	Framework CSS	UI responsive, componentes pre- diseñados, grid system, iconos, compatible con todos los navegadores
JavaScript	ES6+	-	Interactividad cliente	Lenguaje estándar de navegadores, soporte async/await, manipulación DOM
jQuery	3.7.0	MIT	Librería JS	Simplifica manipulación DOM, AJAX, eventos, amplia compatibilidad
SignalR Client (JS)	8.0	MIT	Cliente WebSocket	Cliente JavaScript para conexión con SignalR Hub, manejo de reconexiones
Bootstrap Icons	1.11	MIT	Iconografía	Iconos vectoriales, ligeros, consistentes con Bootstrap

Base de Datos

Tecnología	Versión	Licencia	Propósito	Justificación
SQL Server	2019+	Comercial	RDBMS	Motor de base de datos empresarial, robustez, soporte de Microsoft, herramientas de administración, backups automáticos
T-SQL	-	-	Lenguaje de consultas	Extensión de SQL estándar, stored procedures, triggers, funciones

Infraestructura y Despliegue

Tecnología	Versión	Licencia	Propósito	Justificación
IIS	10.0+	-	Servidor web	Servidor web de Microsoft, integración con Windows Server, soporte para .NET nativo
Kestrel	8.0	MIT	Servidor HTTP	Servidor web multiplataforma de .NET, alto rendimiento, usado internamente por ASP.NET Core
Azure App Service	-	Comercial (PaaS)	Hosting cloud (opcional)	PaaS de Microsoft, escalado automático, integración con Azure SQL, CI/CD integrado
Windows Server	2019+	Comercial	Sistema Operativo	SO estable, compatible con IIS y SQL Server, soporte empresarial

Servicios Externos / APIs

Servicio	Versión	Tipo	Propósito	Justificación
Google Maps API	v3	Comercial (Freemium)	Mapas y geolocalización	API estable, amplia documentación, mapas precisos,

		um)		geocodificación
[Futuro] Stripe/PayPal	-	Comercial	Pasarela de pagos	Integración de pagos en línea (no en v1.0)

Herramientas de Desarrollo

Herramienta	Versión	Licencia	Propósito
Visual Studio	2022+	Community (gratuita) / Professional	IDE principal
Visual Studio Code	Latest	MIT	Editor de código alternativo
SQL Server Management Studio (SSMS)	19+	Gratuita	Gestión de BD
Git	2.40+	GPL	Control de versiones
GitHub	-	Freemium	Repositorio remoto, colaboración
Postman	Latest	Freemium	Testing de APIs
Browser DevTools	-	-	Debugging frontend

2.2.2 Versiones Exactas de Paquetes NuGet Principales

Paquetes instalados en el proyecto:

```
<PackageReference
Include="Microsoft.AspNetCore.Identity.EntityFrameworkCore"
Version="8.0.0" />
<PackageReference Include="Microsoft.EntityFrameworkCore" Version="8.0.0"
/>
<PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer"
Version="8.0.0" />
<PackageReference Include="Microsoft.EntityFrameworkCore.Tools"
Version="8.0.0" />
<PackageReference Include="Microsoft.AspNetCore.SignalR" Version="8.0.0"
/>
<PackageReference Include="Microsoft.Extensions.Http" Version="8.0.0" />
```

2.2.3 Comparación de Alternativas Tecnológicas

Alternativa 1: Backend Framework

Criterio	ASP.NET Core 8 (Elegido)	Node.js + Express	Django (Python)	Spring Boot (Java)
Rendimiento	(Excelente)	(Muy bueno)	(Bueno)	(Muy bueno)
Productividad	(Razor Pages rápido)			
Ecosistema	(Microsoft completo)			
Curva de aprendizaje	(Media)	(Fácil)	(Media)	(Alta)
Soporte empresarial	(Microsoft oficial)	(Comunidad)	(Comunidad)	(Oracle/Spring)
Tiempo real (WebSockets)	(SignalR nativo)	(Socket.io)	(Channels)	(WebSocket API)
Decisión	ELEGIDO	No	No	No

Razón de elección: Familiaridad del equipo con .NET, integración nativa con SQL Server e Identity, alto rendimiento, soporte oficial de Microsoft.

Alternativa 2: Frontend Framework

Criterio	Razor Pages (Elegido)	React SPA	Angular SPA	Blazor WebAssembly
Productividad inicial	(Rápida)	(Configuración)	(Compleja)	(Media)
Experiencia de usuario	(Tradicional)	(SPA fluida)		
SEO	(Server-side)	(Requiere SSR)		
Complejidad	(Baja)	(Alta)	(Muy alta)	(Alta)
Tiempo de desarrollo	(Rápido)	(Medio)	(Lento)	(Medio)
Tamaño del equipo	1-2 devs (Full-stack)	Frontend + Backend	Frontend + Backend	1-2 devs (Full-stack)
Decisión	ELEGIDO	No	No	No

Razón de elección: Desarrollo rápido, un solo lenguaje (C#), no requiere equipo frontend separado, suficiente para v1.0 (responsive con Bootstrap).

Alternativa 3: Base de Datos

Criterio	SQL Server (Elegido)	PostgreSQL	MySQL	MongoDB
----------	-------------------------	------------	-------	---------

Robustez empresarial				
Soporte de Microsoft		(Comunidad)	(Oracle)	(Comunidad)
Integración con .NET	(Nativa)	(Npgsql)	(MySQL.Data)	(Driver)
Herramientas de gestión	(SSMS)	(pgAdmin)	(Workbench)	(Compass)
Costo				
Relacional vs NoSQL	Relacional (RDBMS)	Relacional	Relacional	NoSQL (Documentos)
Decisión	ELEGIDO	No	No	No

Razón de elección: Integración perfecta con .NET y Azure, familiaridad del equipo, SSMS robusto, soporte empresarial de Microsoft. Nota: PostgreSQL sería una excelente alternativa gratuita.

2.2.4 Trade-offs y Decisiones Técnicas

Decisión 1: Server-Side Rendering (Razor Pages) vs Client-Side SPA

Elegido: Server-Side Rendering (Razor Pages)

Trade-offs:

Aspecto	Server-Side (Razor)	Client-Side (SPA)
Ventajas	SEO inmediato, Desarrollo rápido, Menor complejidad, Funciona sin JS	UX más fluida, Menos carga del servidor, Interactividad rica
Desventajas	Recarga de página, Menos interactividad	SEO complejo, Mayor tamaño inicial, Requiere JS habilitado

Razón: Para v1.0, la simplicidad y rapidez de desarrollo son más importantes que una UX ultra-fluida. Se complementa con SignalR para tiempo real donde es crítico (geolocalización).

Decisión 2: Monolito vs Microservicios

Elegido: Arquitectura Monolítica (Single Deployment)

Trade-offs:

Aspecto	Monolito	Microservicios
Ventajas	Simplicidad, Deployment único, Debugging fácil, Menos overhead	Escalado independiente, Tecnologías heterogéneas, Fallos aislados

Desventajas	Escalado vertical, Coupling mayor	Complejidad operativa, Comunicación entre servicios, Debugging difícil
--------------------	-----------------------------------	--

Razón: Para una aplicación de tamaño medio con un equipo pequeño, un monolito bien estructurado es más eficiente. Escalable a microservicios en versiones futuras si es necesario.

Decisión 3: EF Core vs Dapper vs ADO.NET

Elegido: Entity Framework Core

Trade-offs:

Aspecto	EF Core	Dapper	ADO.NET
Ventajas	Productividad alta, Migraciones automáticas, LINQ	Rendimiento excelente, Control fino	Control total, Sin overhead
Desventajas	Overhead ORM, Queries complejas lentas	No genera DDL, Más código manual	Muy verbos, Propenso a errores

Razón: EF Core ofrece el mejor balance entre productividad y rendimiento para aplicaciones CRUD. Para queries críticas se puede usar SQL raw si es necesario.

2.3 Requerimientos de infraestructura y entornos

2.3.1 Hardware Mínimo y Recomendado (On-Premise)

Servidor de Aplicación (Web Server)

Componente	Mínimo	Recomendado	Producción (Alta Disponibilidad)
CPU	2 cores, 2.0 GHz	4 cores, 2.5 GHz	8 cores, 3.0 GHz
RAM	4 GB	8 GB	16 GB
Disco	20 GB SSD	50 GB SSD	100 GB SSD (RAID 1)
Red	100 Mbps	1 Gbps	1 Gbps (redundante)
Sistema Operativo	Windows Server 2019	Windows Server 2022	Windows Server 2022 Datacenter
Software	IIS 10, .NET 8 Runtime	IIS 10, .NET 8 Runtime +	IIS 10, .NET 8 Runtime, monitoreo

	e	SDK	
--	---	-----	--

Servidor de Base de Datos (SQL Server)

Componente	Mínimo	Recomendado	Producción (Alta Disponibilidad)
CPU	2 cores, 2.0 GHz	4 cores, 2.5 GHz	8 cores, 3.0 GHz
RAM	8 GB	16 GB	32 GB
Disco	50 GB SSD	100 GB SSD	500 GB SSD (RAID 10)
Red	100 Mbps	1 Gbps	1 Gbps (redundante)
Sistema Operativo	Windows Server 2019	Windows Server 2022	Windows Server 2022 Datacenter
Software	SQL Server 2019 Express	SQL Server 2019 Standard	SQL Server 2019 Enterprise (Always On)

Notas: - Para desarrollo/pruebas: Servidor único con 8 GB RAM y 100 GB SSD es suficiente - Para producción: Separar servidores de aplicación y base de datos - Backups: Disco adicional de al menos 500 GB para respaldos.

2.3.2 Requerimientos Cloud (Azure - Recomendado)

Opción 1: Ambiente de Desarrollo/Pruebas

Servicio	SKU	Especificaciones	Costo Estimado (USD/mes)
Azure App Service	B1 (Basic)	1 core, 1.75 GB RAM	~\$55
Azure SQL Database	Basic (5 DTU)	2 GB almacenamiento	~\$5
Application Insights	Free tier	1 GB telemetría/mes	\$0
Dominio	.azurewebsites.net	Gratuito	\$0
SSL	Azure Managed Certificate	Gratuito	\$0
Total mensual			~\$60

Opción 2: Ambiente de Producción

Servicio	SKU	Especificaciones	Costo Estimado (USD/mes)
----------	-----	------------------	--------------------------

Azure App Service	S1 (Standard)	1 core, 1.75 GB RAM, auto-scale	~\$75
Azure SQL Database	Standard S0	10 DTU, 250 GB	~\$30
Application Insights	Pay-as-you-go	5 GB telemetría/mes	~\$10
Dominio personalizado	.com + DNS Zone	Registro + hosting DNS	~\$15
SSL	Azure Managed Certificate	Gratuito	\$0
Backup Storage	LRS (50 GB)	RespalDOS automáticos	~\$5
Total mensual			~\$135

Ventajas de Azure: Escalado automático, Backups automáticos, Monitoreo integrado (Application Insights), CI/CD con GitHub Actions, Sin mantenimiento de hardware, SSL gratuito.

2.3.3 Diagrama de Red y Topología de Seguridad

[INSTRUCCIONES PARA DIAGRAMA]

Título: Topología de Red - Transportes Genesis (Producción)

Zonas:

1. ****Zona Pública (DMZ)****
 - Load Balancer / Azure Traffic Manager
 - Firewall / Azure Application Gateway
 - Certificado SSL/TLS
 - IP pública estática

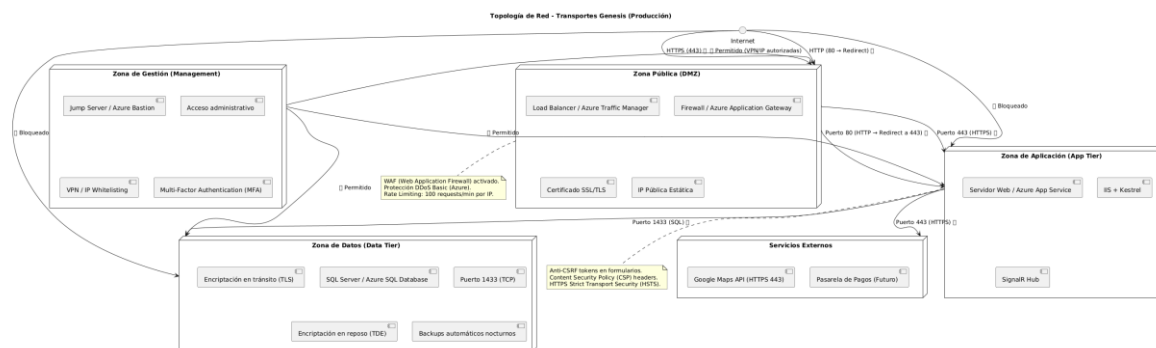
2. ****Zona de Aplicación (App Tier)****
 - Servidor Web / Azure App Service
 - IIS + Kestrel
 - SignalR Hub
 - Sin acceso directo desde internet (solo via Load Balancer)
3. ****Zona de Datos (Data Tier)****
 - SQL Server / Azure SQL Database
 - Puerto 1433 (solo accesible desde App Tier)
 - Encriptación en tránsito (TLS)
 - Encriptación en reposo (TDE - Transparent Data Encryption)
 - Backups automáticos nocturnos
4. ****Zona de Gestión (Management)****
 - Jump Server / Azure Bastion
 - Acceso administrativo
 - VPN o IP whitelisting
 - Multi-Factor Authentication (MFA)

Reglas de Firewall:

- Internet → Load Balancer: Puerto 443 (HTTPS) Permitido
- Internet → Load Balancer: Puerto 80 (HTTP) → Redirect a 443
- Load Balancer → App Tier: Puerto 443 Permitido
- App Tier → Data Tier: Puerto 1433 (SQL) Permitido
- App Tier → Google Maps API: Puerto 443 (HTTPS) Permitido
- Internet → App Tier: Bloqueado (bypass del Load Balancer)
- Internet → Data Tier: Bloqueado
- Management → Todos: Permitido (solo desde VPN/IPs autorizadas)

Seguridad:

- WAF (Web Application Firewall) activado
- DDoS Protection Basic (Azure)
- Rate Limiting: 100 requests/min por IP
- Anti-CSRF tokens en formularios
- Content Security Policy (CSP) headers
- HTTPS Strict Transport Security (HSTS)



2.3.4 Entornos de Desarrollo, Pruebas y Producción

Ambiente de Desarrollo (Local)

Componente	Configuración
IDE	Visual Studio 2022+ Community
Base de Datos	SQL Server 2019 Express (LocalDB)
Servidor Web	IIS Express (incluido en Visual Studio)
Puerto	https://localhost:7XXX
Datos	Datos de prueba (seeders)
Logs	Console output, Debug window

Comandos de inicio para inicializar paquetes, aplicar migraciones y ejecutar aplicacion:

```
# Restaurar paquetes NuGet  
dotnet restore
```

```
# Aplicar migraciones  
dotnet ef database update
```

```
# Ejecutar aplicación  
dotnet run
```

Ambiente de Pruebas (Staging)

Componente	Configuración
Hosting	Azure App Service (B1) o servidor de pruebas
Base de Datos	SQL Server (copia de producción con datos anonimizados)
URL	https://staging.transportesgenesis.com
Datos	Datos de prueba + datos anonimizados de producción
Logs	Application Insights
Propósito	Testing de integración, UAT (User Acceptance Testing)

Despliegue: - CI/CD automático desde rama dev en GitHub - Tests automatizados antes del deploy - Notificación al equipo de QA

Ambiente de Producción

Componente	Configuración
Hosting	Azure App Service (S1+) o servidores dedicados
Base de Datos	SQL Server Standard/Enterprise con Always On
URL	https://app.transportesgenesis.com
Datos	Datos reales de clientes
Logs	Application Insights + Log Analytics
Monitoreo	Azure Monitor, alertas configuradas
Backups	Diarios (automáticos), retención 30 días
Disponibilidad	99.9% SLA (8.76 horas downtime/año)

Despliegue: - Manual o CI/CD desde rama main con aprobación - Blue-Green deployment para cero downtime - Rollback automático si fallan health checks

2.3.5 Especificación de Contenedores (Docker) - Opcional

Aunque no es obligatorio en v1.0, el sistema puede contenerizarse para facilitar despliegue:

Dockerfile (ASP.NET Core App)

Imagen base

FROM mcr.microsoft.com/dotnet/aspnet:8.0 **AS** base

WORKDIR /app

EXPOSE 80

EXPOSE 443

Build

FROM mcr.microsoft.com/dotnet/sdk:8.0 **AS** build

WORKDIR /src

COPY ["TransportesGenesis.csproj", "./"]

RUN dotnet restore

COPY . .

RUN dotnet build -c Release -o /app/build

Publish

FROM build **AS** publish

RUN dotnet publish -c Release -o /app/publish

Final

FROM base **AS** final

WORKDIR /app

COPY --from=publish /app/publish .

ENTRYPOINT ["dotnet", "TransportesGenesis.dll"]

docker-compose.yml

version: '3.8'

services:

web:

build: .

ports:

- "8080:80"
- "8443:443"

environment:

- ASPNETCORE_ENVIRONMENT=Production
-

ConnectionString__DefaultConnection=Server=db;Database=TransportesGenesis;User=sa;Password=YourPassword123!

depends_on:

- db

networks:

- transportesgenesis-network

db:

image: mcr.microsoft.com/mssql/server:2019-latest

environment:

- ACCEPT_EULA=Y
- SA_PASSWORD=YourPassword123!

ports:

- "1433:1433"

volumes:

- sqldata:/var/opt/mssql

networks:

- transportesgenesis-network

volumes:

sqldata:

networks:

transportesgenesis-network:

driver: bridge

Comandos Docker:

Build de imagen

docker build -t transportesgenesis:latest .

Ejecutar contenedores

docker-compose up -d

Ver Logs

docker-compose logs -f web

```
# Detener
```

```
docker-compose down
```

2.3.6 CI/CD Pipeline Básico (GitHub Actions)

Archivo .github/workflows/deploy.yml (Ejemplo)

```
name: Deploy to Azure
```

```
on:
```

```
  push:
```

```
    branches: [ main ]
```

```
  workflow_dispatch:
```

```
jobs:
```

```
  build-and-deploy:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - uses: actions/checkout@v3
```

```
      - name: Setup .NET
```

```
        uses: actions/setup-dotnet@v3
```

```
        with:
```

```
          dotnet-version: '8.0.x'
```

```
      - name: Restore dependencies
```

```
        run: dotnet restore
```

```
      - name: Build
```

```
        run: dotnet build --configuration Release --no-restore
```

```
      - name: Test
```

```
        run: dotnet test --no-restore --verbosity normal
```

```
      - name: Publish
```

```
        run: dotnet publish -c Release -o ./publish
```

```
      - name: Deploy to Azure App Service
```

```
        uses: azure/webapps-deploy@v2
```

```
        with:
```

```
          app-name: 'transportesgenesis-prod'
```

```
          publish-profile: '${{ secrets.AZURE_WEBAPP_PUBLISH_PROFILE }}
```

```
          package: ./publish
```

Flujo: 1. Push a rama main → Trigger automático 2. Checkout del código 3. Build del proyecto 4. Ejecución de tests 5. Publish de artefactos 6. Deploy a Azure App Service 7. Notificación (Slack/Email)

2.4 Diseño de base de datos

2.4.1 Esquema Físico - Tablas Principales

El sistema utiliza **dos esquemas** en SQL Server: - **genesis**: Tablas de negocio (buses, rutas, alumnos, pagos, etc.) - **dbo**: Tablas de ASP.NET Core Identity (usuarios, roles, claims)

Tabla: genesis.Bus

```
CREATE TABLE genesis.Bus (  
    IdBus INT PRIMARY KEY IDENTITY(1,1),  
    Placa NVARCHAR(20) NOT NULL UNIQUE,  
    Modelo NVARCHAR(100),  
    Capacidad INT NOT NULL,  
    Activo TINYINT NOT NULL DEFAULT 1,  
    FechaRegistro DATETIME2 NOT NULL DEFAULT GETDATE(),  
    FechaActualizacion DATETIME2 NULL,  
    UsuarioRegistro NVARCHAR(450) NULL  
);  
  
CREATE INDEX IX_Bus_Placa ON genesis.Bus(Placa);  
CREATE INDEX IX_Bus_Activo ON genesis.Bus(Activo) WHERE Activo = 1;
```

Descripción: Almacena los buses de la empresa.

Tabla: genesis.Ruta

```
CREATE TABLE genesis.Ruta (  
    IdRuta INT PRIMARY KEY IDENTITY(1,1),  
    IdBus INT NOT NULL,  
    Nombre NVARCHAR(200) NOT NULL,  
    Descripcion NVARCHAR(500) NULL,  
    TipoRuta NVARCHAR(20) NOT NULL, -- 'Mañana' o 'Tarde'  
    HoraInicio TIME NOT NULL,  
    EsActiva BIT NOT NULL DEFAULT 1,  
    FechaCreacion DATETIME2 NOT NULL DEFAULT GETDATE(),  
    Activo TINYINT NOT NULL DEFAULT 1,  
    FechaRegistro DATETIME2 NOT NULL DEFAULT GETDATE(),  
  
    CONSTRAINT FK_Ruta_Bus FOREIGN KEY (IdBus) REFERENCES  
genesis.Bus(IdBus),  
    CONSTRAINT CHK_TipoRuta CHECK (TipoRuta IN ('Mañana', 'Tarde'))  
);  
  
CREATE INDEX IX_Ruta_IdBus ON genesis.Ruta(IdBus);  
CREATE INDEX IX_Ruta_TipoRuta ON genesis.Ruta(TipoRuta);  
CREATE INDEX IX_Ruta_EsActiva ON genesis.Ruta(EsActiva) WHERE EsActiva =  
1;
```

Descripción: Almacena las rutas calculadas para cada bus.

Tabla: genesis.Parada

```
CREATE TABLE genesis.Parada (  
    IdParada INT PRIMARY KEY IDENTITY(1,1),  
    Nombre NVARCHAR(200) NOT NULL,  
    Direccion NVARCHAR(500) NULL,  
    Latitud DECIMAL(10,7) NOT NULL,  
    Longitud DECIMAL(10,7) NOT NULL,  
    Activo TINYINT NOT NULL DEFAULT 1,  
    FechaRegistro DATETIME2 NOT NULL DEFAULT GETDATE(),  
    FechaActualizacion DATETIME2 NULL  
);  
  
CREATE INDEX IX_Parada_Coordenadas ON genesis.Parada(Latitud, Longitud);  
CREATE INDEX IX_Parada_Activo ON genesis.Parada(Activo) WHERE Activo = 1;
```

Descripción: Ubicaciones geográficas donde se recogen/dejan alumnos.

Tabla: genesis.RutaParada (Tabla intermedia)

```
CREATE TABLE genesis.RutaParada (  
    IdRutaParada INT PRIMARY KEY IDENTITY(1,1),  
    IdRuta INT NOT NULL,  
    IdParada INT NOT NULL,  
    Orden INT NOT NULL,  
    HoraEstimada TIME NULL,  
    Activo TINYINT NOT NULL DEFAULT 1,  
  
    CONSTRAINT FK_RutaParada_Ruta FOREIGN KEY (IdRuta) REFERENCES  
genesis.Ruta(IdRuta) ON DELETE CASCADE,  
    CONSTRAINT FK_RutaParada_Parada FOREIGN KEY (IdParada) REFERENCES  
genesis.Parada(IdParada),  
    CONSTRAINT UQ_RutaParada_RutaOrden UNIQUE (IdRuta, Orden)  
);  
  
CREATE INDEX IX_RutaParada_IdRuta ON genesis.RutaParada(IdRuta);  
CREATE INDEX IX_RutaParada_Orden ON genesis.RutaParada(IdRuta, Orden);
```

Descripción: Relación many-to-many entre Ruta y Parada, con orden secuencial.

Tabla: genesis.Alumnos

```
CREATE TABLE genesis.Alumnos (  
    IdAlumno INT PRIMARY KEY IDENTITY(1,1),  
    NombreCompleto NVARCHAR(200) NOT NULL,  
    FechaNacimiento DATE NULL,
```



```

Grado NVARCHAR(50) NULL,
IdPadre NVARCHAR(450) NOT NULL, -- FK a AspNetUsers
IdParada INT NULL, -- Parada donde se recoge
Activo TINYINT NOT NULL DEFAULT 1,
FechaRegistro DATETIME2 NOT NULL DEFAULT GETDATE(),

CONSTRAINT FK_Alumnos_Padre FOREIGN KEY (IdPadre) REFERENCES
dbo.AspNetUsers(Id),
CONSTRAINT FK_Alumnos_Parada FOREIGN KEY (IdParada) REFERENCES
genesis.Parada(IdParada)
);

CREATE INDEX IX_Alumnos_IdPadre ON genesis.Alumnos(IdPadre);
CREATE INDEX IX_Alumnos_IdParada ON genesis.Alumnos(IdParada);
CREATE INDEX IX_Alumnos_Activo ON genesis.Alumnos(Activo) WHERE Activo =
1;

```

Descripción: Información de alumnos que usan el servicio de transporte.

Tabla: genesis.AsignacionPilotoBus

```

CREATE TABLE genesis.AsignacionPilotoBus (
    IdAsignacion INT PRIMARY KEY IDENTITY(1,1),
    IdUsuarioPiloto NVARCHAR(450) NOT NULL, -- FK a AspNetUsers
    IdBus INT NOT NULL,
    FechaAsignacion DATETIME2 NOT NULL DEFAULT GETDATE(),
    FechaFinAsignacion DATETIME2 NULL,
    EsActual BIT NOT NULL DEFAULT 1,
    Activo TINYINT NOT NULL DEFAULT 1,
    FechaRegistro DATETIME2 NOT NULL DEFAULT GETDATE(),

    CONSTRAINT FK_AsignacionPilotoBus_Usuario FOREIGN KEY
(IdUsuarioPiloto) REFERENCES dbo.AspNetUsers(Id),
    CONSTRAINT FK_AsignacionPilotoBus_Bus FOREIGN KEY (IdBus) REFERENCES
genesis.Bus(IdBus)
);

CREATE INDEX IX_AsignacionPilotoBus_IdUsuarioPiloto ON
genesis.AsignacionPilotoBus(IdUsuarioPiloto);
CREATE INDEX IX_AsignacionPilotoBus_EsActual ON
genesis.AsignacionPilotoBus(IdUsuarioPiloto, EsActual)
WHERE EsActual = 1;

```

Descripción: Asignación de pilotos/monitores a buses (histórico con campo EsActual).

Tabla: *genesis.RegistroRecogida*

```
CREATE TABLE genesis.RegistroRecogida (  
    IdRegistro INT PRIMARY KEY IDENTITY(1,1),  
    IdParada INT NOT NULL,  
    IdAlumno INT NOT NULL,  
    FechaHoraRecogida DATETIME2 NOT NULL DEFAULT GETDATE(),  
    ConfirmadoPor NVARCHAR(450) NULL, -- FK a AspNetUsers (Monitor)  
    Latitud DECIMAL(10,7) NULL,  
    Longitud DECIMAL(10,7) NULL,  
    AlumnoPresente BIT NOT NULL DEFAULT 1,  
    Activo TINYINT NOT NULL DEFAULT 1,  
    FechaRegistro DATETIME2 NOT NULL DEFAULT GETDATE(),  
  
    CONSTRAINT FK_RegistroRecogida_Parada FOREIGN KEY (IdParada)  
REFERENCES genesis.Parada(IdParada),  
    CONSTRAINT FK_RegistroRecogida_Alumno FOREIGN KEY (IdAlumno)  
REFERENCES genesis.Alumnos(IdAlumno),  
    CONSTRAINT FK_RegistroRecogida_Monitor FOREIGN KEY (ConfirmadoPor)  
REFERENCES dbo.AspNetUsers(Id)  
);  
  
CREATE INDEX IX_RegistroRecogida_IdParada ON  
genesis.RegistroRecogida(IdParada);  
CREATE INDEX IX_RegistroRecogida_IdAlumno ON  
genesis.RegistroRecogida(IdAlumno);  
CREATE INDEX IX_RegistroRecogida_FechaHora ON  
genesis.RegistroRecogida(FechaHoraRecogida);  
CREATE UNIQUE INDEX IX_RegistroRecogida_ParadaAlumno ON  
genesis.RegistroRecogida(IdParada, IdAlumno, FechaHoraRecogida);
```

Descripción: Registros de qué alumnos fueron recogidos en cada parada.

Tabla: *genesis.ConfirmacionAsistencia*

```
CREATE TABLE genesis.ConfirmacionAsistencia (  
    IdConfirmacion INT PRIMARY KEY IDENTITY(1,1),  
    IdAlumno INT NOT NULL,  
    Fecha DATE NOT NULL,  
    VaAsistir BIT NOT NULL,  
    FechaHoraConfirmacion DATETIME2 NOT NULL DEFAULT GETDATE(),  
    Activo TINYINT NOT NULL DEFAULT 1,  
    FechaRegistro DATETIME2 NOT NULL DEFAULT GETDATE(),  
  
    CONSTRAINT FK_ConfirmacionAsistencia_Alumno FOREIGN KEY (IdAlumno)  
REFERENCES genesis.Alumnos(IdAlumno),  
    CONSTRAINT UQ_ConfirmacionAsistencia_AlumnoFecha UNIQUE (IdAlumno,  
Fecha)  
);  
  
CREATE INDEX IX_ConfirmacionAsistencia_Fecha ON  
genesis.ConfirmacionAsistencia(Fecha);
```

```
CREATE INDEX IX_ConfirmacionAsistencia_IdAlumno ON
genesis.ConfirmacionAsistencia(IdAlumno);
```

Descripción: Confirmaciones diarias de asistencia por parte de los padres.

Tabla: *genesis.Pago*

```
CREATE TABLE genesis.Pago (
    IdPago INT PRIMARY KEY IDENTITY(1,1),
    IdPadre NVARCHAR(450) NOT NULL, -- FK a AspNetUsers
    Monto DECIMAL(18,2) NOT NULL,
    FechaPago DATETIME2 NOT NULL DEFAULT GETDATE(),
    MetodoPago NVARCHAR(50) NULL, -- 'Efectivo', 'Transferencia', etc.
    Estado NVARCHAR(20) NOT NULL DEFAULT 'Pendiente', -- 'Pendiente',
    'Confirmado'
    Comprobante NVARCHAR(MAX) NULL, -- Ruta de archivo o Base64
    Activo TINYINT NOT NULL DEFAULT 1,
    FechaRegistro DATETIME2 NOT NULL DEFAULT GETDATE(),

    CONSTRAINT FK_Pago_Padre FOREIGN KEY (IdPadre) REFERENCES
    dbo.AspNetUsers(Id),
    CONSTRAINT CHK_Pago_Estado CHECK (Estado IN ('Pendiente',
    'Confirmado'))
);

CREATE INDEX IX_Pago_IdPadre ON genesis.Pago(IdPadre);
CREATE INDEX IX_Pago_FechaPago ON genesis.Pago(FechaPago);
CREATE INDEX IX_Pago_Estado ON genesis.Pago(Estado);
```

Descripción: Pagos realizados por los padres de familia.

2.4.2 Diagrama Entidad-Relación (ER)

[INSTRUCCIONES PARA DIAGRAMA]

Título: Modelo Entidad-Relación - Transportes Genesis

Entidades Principales:

1. AspNetUsers (dbo) [Usuarios del sistema]
 - Id (PK, NVARCHAR(450))
 - UserName
 - Email
 - PasswordHash
 - ...campos de Identity
2. AspNetRoles (dbo) [Roles del sistema]
 - Id (PK, NVARCHAR(450))
 - Name (Administrador, Piloto, Monitor, PadreDeFamilia)

3. Bus (genesis)
 - IdBus (PK)
 - Placa (UNIQUE)
 - Modelo
 - Capacidad
 - Activo
4. Ruta (genesis)
 - IdRuta (PK)
 - IdBus (FK → Bus)
 - Nombre
 - TipoRuta ('Mañana' | 'Tarde')
 - HoraInicio
 - EsActiva
5. Parada (genesis)
 - IdParada (PK)
 - Nombre
 - Direccion
 - Latitud
 - Longitud
 - Activo
6. RutaParada (genesis) [Intermedia]
 - IdRutaParada (PK)
 - IdRuta (FK → Ruta)
 - IdParada (FK → Parada)
 - Orden
 - HoraEstimada
7. Alumnos (genesis)
 - IdAlumno (PK)
 - NombreCompleto
 - FechaNacimiento
 - Grado
 - IdPadre (FK →AspNetUsers)
 - IdParada (FK → Parada)
8. AsignacionPilotoBus (genesis)
 - IdAsignacion (PK)
 - IdUsuarioPiloto (FK →AspNetUsers)
 - IdBus (FK → Bus)
 - FechaAsignacion
 - EsActual
9. RegistroRecogida (genesis)
 - IdRegistro (PK)
 - IdParada (FK → Parada)
 - IdAlumno (FK → Alumnos)

- FechaHoraRecogida
- ConfirmadoPor (FK →AspNetUsers)
- AlumnoPresente

10. ConfirmacionAsistencia (genesis)

- IdConfirmacion (PK)
- IdAlumno (FK → Alumnos)
- Fecha
- VaAsistir
- FechaHoraConfirmacion

11. Pago (genesis)

- IdPago (PK)
- IdPadre (FK →AspNetUsers)
- Monto
- FechaPago
- MetodoPago
- Estado

Relaciones (Cardinalidades):

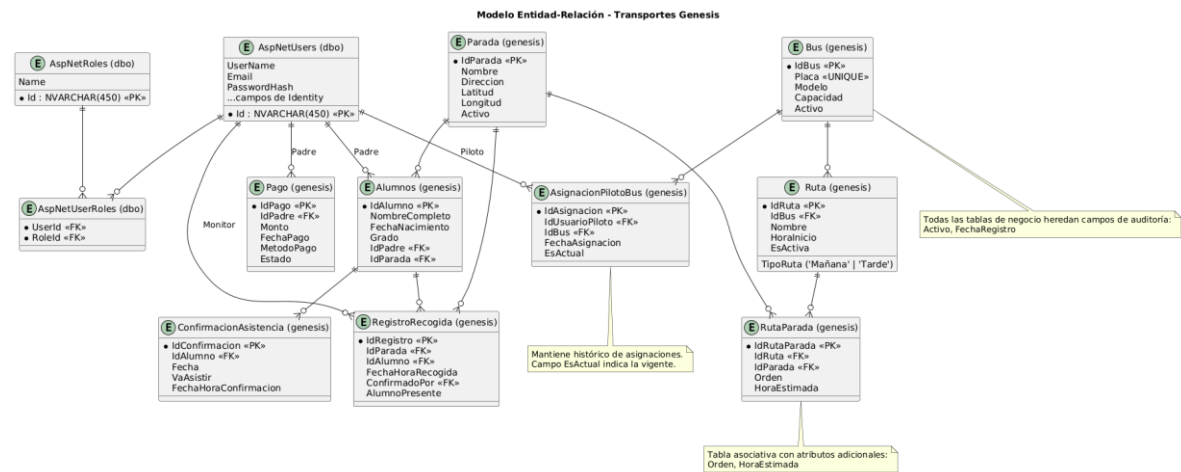
- AspNetUsers (1) ↔ (N) AspNetUserRoles ↔ (N) AspNetRoles [Many-to-Many via tabla intermedia]
- AspNetUsers (1) ↔ (N) Alumnos [Un padre puede tener muchos hijos]
- AspNetUsers (1) ↔ (N) AsignacionPilotoBus [Un piloto puede tener múltiples asignaciones históricas]
- AspNetUsers (1) ↔ (N) RegistroRecogida [Un monitor registra muchas recogidas]
- AspNetUsers (1) ↔ (N) Pago [Un padre realiza muchos pagos]
- Bus (1) ↔ (N) Ruta [Un bus tiene muchas rutas]
- Bus (1) ↔ (N) AsignacionPilotoBus [Un bus puede ser asignado a muchos pilotos]
- Ruta (1) ↔ (N) RutaParada [Una ruta tiene muchas paradas]
- Parada (1) ↔ (N) RutaParada [Una parada puede estar en muchas rutas]
- Parada (1) ↔ (N) Alumnos [Una parada puede tener muchos alumnos asignados]
- Parada (1) ↔ (N) RegistroRecogida [En una parada se registran muchas recogidas]
- Alumnos (1) ↔ (N) RegistroRecogida [Un alumno tiene muchos registros de recogidas]
- Alumnos (1) ↔ (N) ConfirmacionAsistencia [Un alumno tiene muchas confirmaciones]

Notas:

- RutaParada es una tabla asociativa con atributo adicional (Orden,

HoraEstimada)

- AsignacionPilotoBus mantiene histórico con campo "EsActual"
- Todas las tablas de negocio heredan campos de auditoría (Activo, FechaRegistro)



2.4.3 Índices y Estrategias de Optimización

Índices Principales

Tabla	Índice	Tipo	Justificación
Bus	IX_Bus_Placa	UNIQUE	Búsqueda rápida por placa
Ruta	IX_Ruta_IdBus_TipoRuta	COMPOSITE	Filtrado frecuente por bus y turno
RutaParada	IX_RutaParada_IdRuta_Orden	COMPOSITE	Ordenamiento de paradas en una ruta
AsignacionPilotoBus	IX_AsignacionPilotoBus_IdUsuarioPiloto_EsActual	FILTERED	Búsqueda de asignación activa del piloto
RegistroRecogida	IX_RegistroRecogida_ParadaAlumno	UNIQUE	Prevenir registros duplicados
ConfirmacionAsistencia	IX_ConfirmacionAsistencia_AlumnoFecha	UNIQUE	Una confirmación por alumno por día

Estrategias de Particionamiento (Futuro)

Para tablas que crecen rápidamente:

```
-- Particionar RegistroRecogida por año (horizontal partitioning)
CREATE PARTITION FUNCTION PF_RegistroRecogida_Year (DATETIME2)
AS RANGE RIGHT FOR VALUES ('2024-01-01', '2025-01-01', '2026-01-01');
```

```
CREATE PARTITION SCHEME PS_RegistroRecogida_Year  
AS PARTITION PF_RegistroRecogida_Year ALL TO ([PRIMARY]);
```

Beneficio: Mejora de rendimiento en consultas por rango de fechas.

2.4.4 Estrategia de Migración de Datos (si aplica)

Si existe un sistema legacy con datos previos:

Fase 1: Análisis

6. Identificar fuentes de datos (Excel, Access, SQL Server antiguo)
7. Mapear campos antiguos → nuevos
8. Detectar inconsistencias (duplicados, formatos)

Fase 2: Limpieza (ETL)

-- Ejemplo: Migrar buses desde tabla antigua

```
INSERT INTO genesis.Bus (Placa, Modelo, Capacidad, Activo, FechaRegistro)  
SELECT  
    UPPER(TRIM(Placa)),  
    TRIM(Modelo),  
    ISNULL(Capacidad, 40),  
    1,  
    GETDATE()  
FROM SistemaAntiguo.dbo.Buses  
WHERE Placa IS NOT NULL;
```

Fase 3: Validación

- Comparar conteos de registros
- Verificar integridad referencial
- Auditoría de datos críticos (pagos, alumnos)

Fase 4: Cutover

- Backup completo del sistema antiguo
- Migración en horario no laboral (noche/fin de semana)
- Rollback plan preparado

2.5 Diseño de APIs e integración

2.5.1 Endpoints REST Principales

El sistema expone una **API REST** bajo la ruta base `/api/` para operaciones programáticas.

API de Rutas

GET /api/rutas/bus/{idBus}/activa

Obtiene la ruta activa del día para un bus específico.

GET /api/rutas/bus/1/activa?tipoRuta=Mañana HTTP/1.1

Host: app.transportesgenesis.com

Authorization: Cookie (Identity)

Respuesta exitosa (200 OK):

```
{
  "success": true,
  "message": "Ruta encontrada",
  "data": {
    "idRuta": 123,
    "idBus": 1,
    "placaBus": "ABC-123",
    "nombre": "Ruta Mañana - Zona Norte",
    "tipoRuta": "Mañana",
    "horaInicio": "06:30:00",
    "esActiva": true,
    "paradas": [
      {
        "idParada": 10,
        "nombreParada": "Parada Escuela XYZ",
        "latitud": 14.12345,
        "longitud": -87.98765,
        "orden": 1,
        "horaEstimada": "06:45:00",
        "alumnos": [
          {
            "idAlumno": 50,
            "nombreCompleto": "Juan Pérez",
            "grado": "5to Primaria"
          }
        ]
      }
    ]
  },
  "fechaCreacion": "2025-01-20T10:00:00Z"
}
```


Respuesta sin ruta (404 Not Found):

```
{
  "success": false,
  "message": "No hay ruta calculada para este bus en el turno Mañana",
  "data": null
}
```

POST /api/rutas/calcular

Calcula una ruta optimizada usando el algoritmo TSP (Traveling Salesman Problem).

```
POST /api/rutas/calcular HTTP/1.1
Host: app.transportesgenesis.com
Content-Type: application/json
Authorization: Cookie (Identity, Rol: Administrador)
```

```
{
  "idBus": 1,
  "fecha": "2025-01-21",
  "tipoRuta": "Mañana",
  "idsAlumnos": [50, 51, 52, 53]
}
```

Respuesta exitosa (201 Created):

```
{
  "success": true,
  "message": "Ruta calculada exitosamente",
  "data": {
    "idRuta": 124,
    "totalParadas": 4,
    "distanciaTotal": 12.5,
    "tiempoEstimado": "45 minutos"
  }
}
```

API de Pagos

GET /api/pagos/padre/{idPadre}

Obtiene el historial de pagos de un padre de familia.

```
GET /api/pagos/padre/abc123xyz HTTP/1.1
Host: app.transportesgenesis.com
Authorization: Cookie (Identity)
```

Respuesta exitosa (200 OK):

```
{
  "success": true,
  "data": [
```

```

{
  "idPago": 100,
  "monto": 250.00,
  "fechaPago": "2025-01-15T14:30:00Z",
  "metodoPago": "Transferencia",
  "estado": "Confirmado"
},
{
  "idPago": 99,
  "monto": 250.00,
  "fechaPago": "2024-12-15T10:00:00Z",
  "metodoPago": "Efectivo",
  "estado": "Confirmado"
}
]
}

```

POST /api/pagos

Registra un nuevo pago.

POST /api/pagos HTTP/1.1

Host: app.transportesgenesis.com

Content-Type: application/json

Authorization: Cookie (Identity, Rol: PadreDeFamilia)

```

{
  "idPadre": "abc123xyz",
  "monto": 250.00,
  "metodoPago": "Transferencia",
  "comprobante": "data:image/png;base64,iVBOR..."
}

```

Respuesta exitosa (201 Created):

```

{
  "success": true,
  "message": "Pago registrado exitosamente. Pendiente de confirmación.",
  "data": {
    "idPago": 101,
    "estado": "Pendiente"
  }
}

```

API de Confirmación de Asistencia

POST /api/confirmacion-asistencia

Confirma si un alumno asistirá en una fecha específica.

```
POST /api/confirmacion-asistencia HTTP/1.1
Host: app.transportesgenesis.com
Content-Type: application/json
Authorization: Cookie (Identity, Rol: PadreDeFamilia)
```

```
{
  "idAlumno": 50,
  "fecha": "2025-01-22",
  "vaAsistir": true
}
```

Respuesta exitosa (200 OK):

```
{
  "success": true,
  "message": "Asistencia confirmada para el 22/01/2025"
}
```

2.5.2 SignalR Hub - Geolocalización en Tiempo Real

Hub: *GeolocalizacionHub*

Ubicación: Hubs/GeolocalizacionHub.cs

Métodos del servidor (invocados por clientes):

```
public async Task EnviarUbicacion(double latitud, double longitud, int
idBus)
{
    // Validar usuario autenticado
    var userId = Context.UserIdentifier;

    // Broadcast a todos los clientes conectados al grupo "Mapa"
    await Clients.Group("Mapa").SendAsync(
        "RecibirUbicacion",
        latitud,
        longitud,
        idBus,
        DateTime.Now
    );

    // Opcional: Guardar en BD para histórico
    // await _context.HistorialUbicaciones.AddAsync(...);
}

public async Task UnirseAlMapa()
```

```

{
    await Groups.AddToGroupAsync(Context.ConnectionId, "Mapa");
}

public async Task SalirDelMapa()
{
    await Groups.RemoveFromGroupAsync(Context.ConnectionId, "Mapa");
}

```

Métodos del cliente (invocados por servidor):

```

// Cliente JavaScript (en la página del mapa)
connection.on("RecibirUbicacion", function (latitud, longitud, idBus,
timestamp) {
    // Actualizar marcador en el mapa
    actualizarMarcadorBus(idBus, latitud, longitud);
    console.log(`Bus ${idBus} en [${latitud}, ${longitud}] a las
${timestamp}`);
});

```

Flujo de conexión:

1. Cliente se conecta a /geolocalizacionHub
2. Cliente invoca "UnirseAlMapa"
3. Servidor agrega al cliente al grupo "Mapa"
4. Bus envía ubicación cada 10 segundos vía "EnviarUbicacion"
5. Servidor hace broadcast a todos en grupo "Mapa"
6. Clientes reciben "RecibirUbicacion" y actualizan UI
7. Al salir, cliente invoca "SalirDelMapa"

2.5.3 Integración con Servicios Externos

Google Maps API

Propósito: Visualización de mapas y geocodificación.

Configuración:

```

// appsettings.json
{
  "GoogleMaps": {
    "ApiKey": "YOUR_GOOGLE_MAPS_API_KEY_HERE"
  }
}

```

Uso en JavaScript:

```

<script
src="https://maps.googleapis.com/maps/api/js?key=YOUR_API_KEY&libraries=p
laces"></script>

```

```

<script>
function inicializarMapa() {
    var mapa = new google.maps.Map(document.getElementById('map'), {
        center: {lat: 14.0723, lng: -87.1921}, // Tegucigalpa, Honduras
        zoom: 13
    });

    // Agregar marcador de bus
    var marcador = new google.maps.Marker({
        position: {lat: 14.0723, lng: -87.1921},
        map: mapa,
        title: 'Bus #1'
    });
}
</script>

```

Endpoints usados:

- **Maps JavaScript API:** Renderizado de mapas interactivos
- **Geocoding API (opcional):** Convertir direcciones en coordenadas
- **Directions API (futuro):** Calcular rutas óptimas con tráfico en tiempo real

Pasarela de Pagos (Stripe)

Propósito: Pagos en línea con tarjeta de crédito/débito.

Flujo de integración:

1. Padre ingresa datos de tarjeta en formulario
2. Frontend envía datos a Stripe (tokenización)
3. Stripe devuelve token seguro
4. Backend recibe token y procesa cargo
5. Stripe confirma pago
6. Sistema registra pago en BD
7. Usuario recibe confirmación

Ejemplo de código (servidor):

```

// NuGet: Stripe.net
var options = new ChargeCreateOptions
{
    Amount = 25000, // $250.00 en centavos
    Currency = "usd",
    Source = tokenFromFrontend,
    Description = "Pago mensualidad transporte escolar"
};

var service = new ChargeService();
Charge charge = await service.CreateAsync(options);

```

```

if (charge.Status == "succeeded")
{
    // Registrar pago en BD
    var pago = new Pago
    {
        IdPadre = userId,
        Monto = 250.00m,
        MetodoPago = "Tarjeta",
        Estado = "Confirmado"
    };
    _context.Pagos.Add(pago);
    await _context.SaveChangesAsync();
}

```

Estado: No implementado en v1.0 (pagos se registran manualmente).

2.5.4 Contratos de Interfaz (OpenAPI/Swagger)

Especificación OpenAPI 3.0:

```

openapi: 3.0.0
info:
  title: Transportes Genesis API
  version: 1.0.0
  description: API REST para gestión de transporte escolar

servers:
  - url: https://app.transportesgenesis.com/api
    description: Servidor de producción

paths:
  /rutas/bus/{idBus}/activa:
    get:
      summary: Obtener ruta activa de un bus
      parameters:
        - name: idBus
          in: path
          required: true
          schema:
            type: integer
        - name: tipoRuta
          in: query
          required: true
          schema:
            type: string
            enum: [Mañana, Tarde]
      responses:
        '200':
          description: Ruta encontrada
          content:
            application/json:

```

```
        schema:
          $ref: '#/components/schemas/RutaDto'
      '404':
        description: No hay ruta calculada

components:
  schemas:
    RutaDto:
      type: object
      properties:
        idRuta:
          type: integer
        idBus:
          type: integer
        nombre:
          type: string
        tipoRuta:
          type: string
        paradas:
          type: array
          items:
            $ref: '#/components/schemas/ParadaDto'

    ParadaDto:
      type: object
      properties:
        idParada:
          type: integer
        nombreParada:
          type: string
        latitud:
          type: number
          format: double
        longitud:
          type: number
          format: double
        orden:
          type: integer
```

Documentación interactiva (Swagger UI):

Habilitada en desarrollo: <https://localhost:7XXX/swagger>

```
// Startup.cs
public void ConfigureServices(IServiceCollection services)
{
    services.AddSwaggerGen(c =>
    {
        c.SwaggerDoc("v1", new OpenApiInfo
        {
            Title = "Transportes Genesis API",
            Version = "v1"
        });
    });
}

public void Configure(IApplicationBuilder app)
{
    if (env.IsDevelopment())
    {
        app.UseSwagger();
        app.UseSwaggerUI(c =>
        {
            c.SwaggerEndpoint("/swagger/v1/swagger.json", "API v1");
        });
    }
}
```

2.6 Patrones de diseño y buenas prácticas

2.6.1 Patrones de Diseño Aplicados

1. *Page Model Pattern (Razor Pages)*

Descripción: Cada página Razor tiene su propio modelo (PageModel) que encapsula la lógica de esa página.

Ejemplo:

```
// Pages/Piloto/MiRuta.cshtml.cs
public class MiRutaModel : PageModel
{
    private readonly ApplicationDbContext _context;

    public RutaDto RutaActiva { get; set; }

    public async Task OnGetAsync()
    {
        // Lógica para obtener la ruta
        var userId = User.FindFirst(ClaimTypes.NameIdentifier)?.Value;
```



```

        var asignacion = await _context.AsignacionesPilotoBusDb
            .FirstOrDefaultAsync(a => a.IdUsuarioPiloto == userId &&
a.EsActual);
        // ...
    }
}

```

Beneficio: Separación clara entre vista (CSHTML) y lógica (CS).

2. *Dependency Injection (DI)*

Descripción: Las dependencias se inyectan vía constructor, no se instancian manualmente.

Registro en Startup.cs:

```

services.AddScoped<ApplicationDbContext>();
services.AddHttpClient();
services.AddSignalR();

```

Inyección en constructores:

```

public MiRutaModel(
    IHttpConnectionFactory httpClientFactory,
    ApplicationDbContext context)
{
    _httpClientFactory = httpClientFactory;
    _context = context;
}

```

Beneficio: Desacoplamiento, facilita testing (mocking).

3. *Repository Pattern (Implícito con EF Core)*

Descripción: ApplicationDbContext actúa como Unit of Work y cada DbSet<T> como Repository.

Ejemplo:

```

// ApplicationDbContext.cs
public class ApplicationDbContext : IdentityDbContext<AppUser>
{
    public DbSet<Bus> BusesDb { get; set; }
    public DbSet<Ruta> RutasDb { get; set; }
    public DbSet<Parada> ParadasDb { get; set; }
    // ...
}

// Uso en PageModel
var buses = await _context.BusesDb.Where(b => b.Activo ==
1).ToListAsync();

```

Beneficio: Abstracción del acceso a datos, queries con LINQ.

4. DTO (Data Transfer Objects)

Descripción: Objetos para transferir datos entre capas sin exponer entidades de dominio.

Ejemplo:

```
// DTOs/Ruta/RutaDto.cs
public class RutaDto
{
    public int IdRuta { get; set; }
    public string Nombre { get; set; }
    public List<ParadaRutaDto> Paradas { get; set; }
}

// Uso: Entidad → DTO
var rutaDto = new RutaDto
{
    IdRuta = ruta.IdRuta,
    Nombre = ruta.Nombre,
    Paradas = ruta.Paradas.Select(p => new ParadaRutaDto { ...
}).ToList()
};
```

Beneficio: Seguridad (no exponer propiedades internas), flexibilidad.

5. Authorization Pattern

Descripción: Protección de recursos mediante atributos [Authorize].

Ejemplo:

```
[Authorize(Roles = "Piloto")]
public class MiRutaModel : PageModel { }

[Authorize(Roles = "Monitor")]
public class RegistrarRecogidasModel : PageModel { }

[AllowAnonymous]
public class LoginModel : PageModel { }
```

Beneficio: Seguridad centralizada, fácil de auditar.

6. Factory Pattern (IHttpClientFactory)

Descripción: Creación de instancias de HttpClient mediante factory para evitar agotamiento de sockets.

Ejemplo:

```
// Registro
services.AddHttpClient();

// Uso
var client = _httpClientFactory.CreateClient();
client.BaseAddress = new Uri($"{Request.Scheme}://{Request.Host}");
var response = await client.GetAsync("/api/rutas/...");
```

Beneficio: Gestión eficiente de conexiones HTTP.

2.6.2 Principios SOLID Aplicados

Principio	Aplicación en el Proyecto
Single Responsibility	Cada PageModel tiene una responsabilidad única (ej: MiRutaModel solo maneja la vista de ruta)
Open/Closed	Clases base como Auditoria permiten extensión sin modificación
Liskov Substitution	Clases derivadas de Auditoria son intercambiables
Interface Segregation	Interfaces específicas (ej: IHttpClientFactory en lugar de interfaz genérica)
Dependency Inversion	Dependencias a abstracciones (IHttpClientFactory, DbContext), no implementaciones concretas

2.6.3 Estrategia de Testing

Testing Unitario (Unit Tests)

Framework: xUnit + Moq

Ejemplo de test:

```
public class MiRutaModelTests
{
    [Fact]
    public async Task
    OnGetAsync_ConAsignacionActiva_ReturnaRutaCorrecta()
    {
        // Arrange
        var options = new DbContextOptionsBuilder<ApplicationDbContext>()
            .UseInMemoryDatabase("TestDb")
            .Options;

        var context = new ApplicationDbContext(options);
        var model = new MiRutaModel(null, context);

        // Act
```

```

        await model.OnGetAsync();

        // Assert
        Assert.NotNull(model.RutaActiva);
    }
}

```

Cobertura objetivo: > 70% de cobertura de código

Testing de Integración

Framework: ASP.NET Core TestServer

Ejemplo:

```

public class RutasApiTests :
    IClassFixture<WebApplicationFactory<Startup>>
{
    private readonly HttpClient _client;

    public RutasApiTests(WebApplicationFactory<Startup> factory)
    {
        _client = factory.CreateClient();
    }

    [Fact]
    public async Task GetRutaActiva_RetornaRutaExistente()
    {
        // Act
        var response = await
        _client.GetAsync("/api/rutas/bus/1/activa?tipoRuta=Mañana");

        // Assert
        response.EnsureSuccessStatusCode();
        var content = await response.Content.ReadAsStringAsync();
        Assert.Contains("\"success\":true", content);
    }
}

```

Testing End-to-End (E2E)

Herramienta: Selenium WebDriver o Playwright

Ejemplo de escenario:

1. Usuario navega a /Auth/Login
2. Ingresa credenciales de piloto
3. Click en "Iniciar Sesión"
4. Verifica redirección a /Piloto/MiRuta
5. Verifica que aparece "Bus #1" en la página.

2.6.4 Buenas Prácticas de Código

Convenciones de Nomenclatura

// PascalCase para clases, métodos, propiedades

```
public class Bus { }  
public void CalcularRuta() { }  
public string NombreCompleto { get; set; }
```

// camelCase para parámetros, variables locales

```
public void AsignarBus(int idBus, string usuarioPiloto)  
{  
    var asignacion = new AsignacionPilotoBus();  
}
```

// _camelCase para campos privados

```
private readonly ApplicationDbContext _context;
```

// UPPERCASE para constantes

```
public const string TIPO_RUTA_MANANA = "Mañana";
```

Async/Await

// Correcto: métodos asíncronos con sufijo Async

```
public async Task<RutaDto> OnGetAsync()  
{  
    var ruta = await _context.RutasDb.FirstOrDefaultAsync();  
    return ruta;  
}
```

// Incorrecto: no usar .Result o .Wait() (bloquea thread)

```
var ruta = _context.RutasDb.FirstOrDefaultAsync().Result; // NO HACER
```

Validación de Entrada

// Validar parámetros en controllers/pages

```
if (string.IsNullOrEmpty(IdPiloto))  
{  
    MensajeError = "Usuario no identificado";  
    return Page();  
}
```

// Usar Data Annotations en modelos

```
public class LoginViewModel  
{  
    [Required(ErrorMessage = "El email es requerido")]  
    [EmailAddress]  
    public string Email { get; set; }  
  
    [Required]  
    [DataType(DataType.Password)]  
    public string Password { get; set; }  
}
```

Logging

// Usar ILogger para registrar eventos

```
private readonly ILogger<MiRutaModel> _logger;

public async Task OnGetAsync()
{
    _logger.LogInformation($"Usuario {IdPiloto} consultó su ruta");

    try
    {
        // ...
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Error al obtener ruta");
    }
}
```

2.7 Plan de implementación y roadmap técnico

2.7.1 Fases del Proyecto (Completadas)

Fase	Duración	Módulos Implementados	Estado	Fecha
Fase 0: Setup Inicial	1 semana	Configuración de proyecto, BD, Identity	COMPLETADO	Abril 2026
Fase 1: MVP Core	4 semanas	Login, Roles, Gestión de usuarios	COMPLETADO	Abril 2026
Fase 2: Rutas	3 semanas	CRUD de buses, paradas, cálculo de rutas (TSP)	COMPLETADO	Abril 2026
Fase 3: Geolocalización	2 semanas	SignalR Hub, Mapa en tiempo real	COMPLETADO	Mayo 2026
Fase 4: Pagos y Asistencia	3 semanas	Módulo de pagos, Confirmación de asistencia	COMPLETADO	Mayo 2026
Fase 5: Área de Monitores	2 semanas	Registro de recogidas, Área del Monitor	COMPLETADO	Mayo 2026
Fase 6: Testing y Deploy	2 semanas	Testing, Documentación, Despliegue	EN CURSO	Mayo 2026

Total: ~17 semanas (4.25 meses)

2.7.2 Roadmap Futuro (v2.0 y posteriores)

v2.0 - Mejoras de UX y Notificaciones (Q2 2025)

Feature	Prioridad	Esfuerzo	Descripción
Notificaciones Push	Alta	3 dias	Integración con Firebase/OneSignal para notificar llegada del bus
Modo offline	Baja	4 dias	PWA con Service Workers para funcionar sin internet

v2.5 - Integraciones y Reportes (Q3 2025)

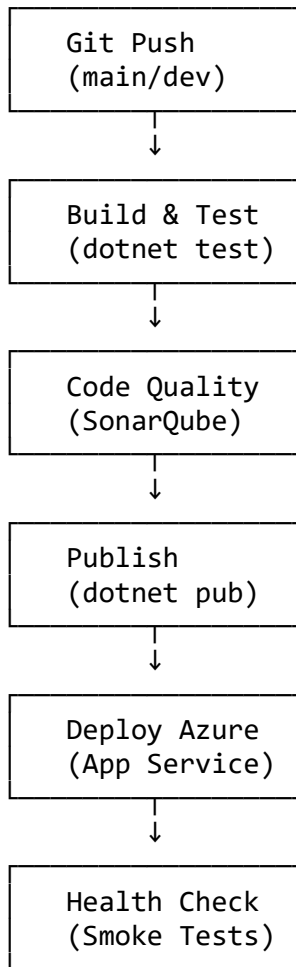
Feature	Prioridad	Esfuerzo	Descripción
Integración con Stripe	Alta	2 dias	Pagos en línea con tarjeta
Reportes avanzados (Power BI)	Media	3 dias	Dashboards interactivos con Power BI Embedded
Facturación electrónica	Media	5 dias	Generación de facturas según regulaciones locales

V3.0 - Machine Learning y Optimización (Q1 2026)

Feature	Prioridad	Esfuerzo	Descripción
Predicción de tiempos	Media	4 semanas	ML para predecir tiempos de llegada con tráfico
Optimización dinámica	Media	5 semanas	Recalcular rutas en tiempo real según tráfico
Análisis de patrones	Baja	3 semanas	Análisis de asistencia, pagos, comportamiento

2.7.3 Estrategia de DevOps

CI/CD Pipeline (GitHub Actions)



Monitoreo (Application Insights)

- **Métricas clave:**
 - Request rate (requests/min)
 - Response time (p50, p95, p99)
 - Error rate (%)
 - Active users
 - SignalR connections
- **Alertas configuradas:**
 - Error rate > 5% → Email a equipo
 - Response time > 2s en 5 minutos → Slack
 - Disk space < 10% → SMS
 - SQL DTU > 90% → Email

Logging (Serilog + Azure Log Analytics)

// Configuración en Startup.cs

```
Log.Logger = new LoggerConfiguration()  
    .MinimumLevel.Information()  
    .WriteTo.Console()  
    .WriteTo.File("logs/app.txt", rollingInterval: RollingInterval.Day)  
    .WriteTo.AzureAnalytics(workspaceId, authenticationId)  
    .CreateLogger();
```

Niveles de log: - **Trace:** Debugging detallado - **Debug:** Información de desarrollo - **Information:** Flujo normal de la aplicación - **Warning:** Situaciones anormales pero manejables - **Error:** Errores que impiden una operación - **Critical:** Fallos catastróficos

2.8 Análisis de riesgos técnicos y plan de mitigación

2.8.1 Matriz de Riesgos Técnicos

ID	Riesgo	Probabilidad	Impacto	Severidad	Mitigación	Contingencia
RT-01	Pérdida de conexión SignalR (WebSocket)	Media (40%)	Alto	 Alta	- Implementar reconexión automática- Fallback a Server-Sent Events- Fallback a Long Polling- Timeout de 30 segundos	- Mostrar mensaje "Reconectando ..." - Permitir uso sin tiempo real (solo refresh manual)
RT-02	Sobrecarga de base de datos con muchos usuarios	Baja (20%)	Alto	 Media	- Índices optimizados en consultas frecuentes- Caching con Redis (futuro)- Connection pooling habilitado- Query optimization con EF Core	- Escalar verticalmente SQL Server- Implementar Azure SQL Database con auto-scaling- Separar BD de lectura y escritura (CQRS)
RT-03	Bug en algoritmo TSP con muchas	Media (30%)	Medio	 Media	- Tests unitarios exhaustivos-	- Calcular ruta de forma manual- Dividir

	paradas (>30)				Límite de 30 paradas por ruta- Timeout de 60 segundos en cálculo- Algoritmo heurístico (nearest neighbor)	ruta en sub-rutas- Usar servicios externos (Google Directions API)
RT-04	Fallo de Google Maps API (cuota excedida)	Baja (15%)	Alto	● Media	- Monitorear uso de API mensualmente- Cachear mapas estáticos- Optimizar llamadas (lazy loading)	- Usar OpenStreetMap como backup- Mostrar coordenadas sin mapa- Aumentar cuota de Google Maps
RT-05	Vulnerabilidades de seguridad (XSS, CSRF, SQL Injection)	Media (30%)	Crítico	● Alta	- Validación de entrada (Data Annotations) - Anti-CSRF tokens automáticos (Razor)- Parametrización de queries (EF Core)- Content Security Policy headers- Auditorías de seguridad trimestrales	- Aplicar parches inmediatamente- Notificar a usuarios- Cambiar contraseñas forzosamente- Contratar auditoría externa
RT-06	Incompatibilidad con navegadores antiguos	Baja (10%)	Bajo	● Baja	- Usar Babel/polyfills para JS- Detección de navegador en frontend-	- Mostrar mensaje: "Actualiza tu navegador"- Versión simplificada

					Avisar a usuarios sobre requisitos	para navegadores viejos
RT-07	Pérdida de datos por fallo de disco	Muy Baja (5%)	Crítico	● Media	- Backups automáticos diarios- Retención de 30 días- Geo-redundancia (Azure)- RAID 10 en on-premise	- Restaurar desde backup más reciente- Máximo pérdida: 24 horas de datos- Proceso de restauración documentado
RT-08	Rendimiento degradado en horas pico	Media (35%)	Medio	● Media	- Load balancer con múltiples instancias- Auto-scaling habilitado- CDN para contenido estático- Output caching en páginas estáticas	- Escalar manualmente más instancias- Priorizar funciones críticas (login, mapa)- Mensaje: “Alta demanda, procesando...”
RT-09	Errores de migración de base de datos	Baja (15%)	Alto	● Media	- Revisar migraciones antes de aplicar- Backup antes de cada migración- Probar en staging primero- Rollback script preparado	- Rollback a versión anterior- Aplicar migración manualmente- Hotfix deployment
RT-10	Dependencia de terceros (Google Maps) sin respuesta	Muy Baja (5%)	Medio	● Baja	- Circuit Breaker pattern- Timeout de	- Degradar funcionalidad (sin mapa)- Mostrar

					5 segundos- Retry con backoff exponencial	coordenadas en texto- Notificar al equipo técnico
--	--	--	--	--	--	--

Leyenda: - ● **Severidad Alta:** Requiere acción inmediata - ● **Severidad Media:** Monitorear de cerca - ● **Severidad Baja:** Aceptable con mitigación básica

2.8.2 Plan de Respuesta a Incidentes

Niveles de Severidad

Nivel	Criterio	Tiempo de Respuesta	Ejemplo
P0 - Crítico	Sistema completamente caído	< 15 minutos	BD inaccesible, servidor caído
P1 - Alto	Funcionalidad crítica no disponible	< 1 hora	Login no funciona, mapa no carga
P2 - Medio	Funcionalidad secundaria afectada	< 4 horas	Reportes lentos, emails no enviados
P3 - Bajo	Bug menor sin impacto operativo	< 24 horas	Formato de fecha incorrecto, typo en texto

Equipo de Respuesta

9. **On-call Engineer:** Monitoreo 24/7 (rotación semanal)
10. **DevOps Lead:** Escalamiento de infraestructura
11. **Database Admin:** Problemas de BD
12. **Product Owner:** Decisiones de negocio

Proceso de Escalamiento

```

Incidente detectado (alerta o reporte)
↓
On-call Engineer investiga (15 min)
↓
¿Puede resolver? → SÍ → Implementa fix → Documenta
↓ NO
Escala a DevOps Lead (30 min)
↓
¿Requiere cambio mayor? → SÍ → Escala a Product Owner
↓ NO
Implementa fix → Testing → Deploy → Postmortem

```

2.8.3 Estrategia de Backup y Recuperación

Backups Automáticos

Tipo	Frecuencia	Retención	Ubicación	Propósito
Full Backup	Diario (2:00 AM)	30 días	Azure Blob Storage	Recuperación completa
Differential	Cada 6 horas	7 días	Azure Blob Storage	Recuperación rápida
Transaction Log	Cada 15 minutos	24 horas	Azure Blob Storage	Punto en el tiempo
Código fuente	Cada commit	Permanente	GitHub	Recuperación de código

Procedimiento de Restauración

Escenario 1: Pérdida de datos recientes (< 24 horas)

-- Restaurar desde backup más reciente

```
RESTORE DATABASE TransportesGenesis  
FROM DISK = 'backup_2025_01_20.bak'  
WITH NORECOVERY;
```

-- Aplicar transaction logs hasta hora específica

```
RESTORE LOG TransportesGenesis  
FROM DISK = 'log_2025_01_20_14_00.trn'  
WITH STOPAT = '2025-01-20 13:45:00';
```

Tiempo estimado: 30 minutos

Pérdida máxima de datos: 15 minutos

Escenario 2: Corrupción de base de datos

13. Identificar tablas corruptas
14. Restaurar desde backup en servidor de pruebas
15. Exportar solo datos corruptos
16. Importar en BD de producción
17. Validar integridad referencial

Tiempo estimado: 2-4 horas

Pérdida máxima de datos: Ninguna (si backup es reciente)

2.8.4 Lessons Learned (Post-Mortem)

Después de cada incidente P0 o P1, se debe realizar un post-mortem:

Plantilla:

Post-Mortem: [Título del Incidente]

Fecha: YYYY-MM-DD

Duración: X horas

Severidad: P0/P1

Afectados: X usuarios

¿Qué pasó?

Descripción breve del incidente.

Línea de Tiempo

- 10:00 AM: Alerta de Application Insights
- 10:05 AM: Investigación inicial
- 10:30 AM: Root cause identificado
- 11:00 AM: Fix aplicado
- 11:15 AM: Sistema estable

Root Cause

Explicación técnica de la causa raíz.

¿Qué salió bien?

- Detección rápida gracias a alertas
- Comunicación efectiva del equipo

¿Qué salió mal?

- Faltó monitoreo en X componente
- Documentación desactualizada

Acción Items

1. ☐ Agregar alertas para X (Responsable: Juan, Fecha: 2025-01-25)
2. ☐ Actualizar documentación (Responsable: María, Fecha: 2025-01-22)
3. ☐ Revisar proceso de deploy (Responsable: Pedro, Fecha: 2025-01-30)